Processor API v7

Installation

Docker

1. Follow the steps here ↗ to install Docker Engine on your host machine.

2. Verify your installation by running `docker run hello-world`

TensorRT (GPU)

Install GPU Driver

> ⚠ This setup guide is for Linux-based machines. For Windows machines, we recommend following the official guide from Docker ↗.

1. Download and run the appropriate NVIDIA driver installer ↗. To use our latest containers, make sure to install a driver compatible with TensorRT 8.6.1. The recommended version is CUDA Toolkit 12.0+.

2. Verify your driver installation by running `nvidia-smi`. The output should start with the following:

```
+-----------------------------------------------------------------------------
----------+
| NVIDIA-SMI 525.125.06      Driver Version: 525.125.06    CUDA
Version: 12.0 |
|-------------------------------+----------------------+------------
----------+
```

Install NVIDIA Container Runtime

1. Install gcc and make

```
sudo apt-get update
sudo apt-get install -y gcc make
```

2. Install NVIDIA Container Runtime

```
distribution=$(. /etc/os-release;echo $ID$VERSION_ID) \
    && curl -s -L https://nvidia.github.io/nvidia-docker/gpgkey |
sudo apt-key add - \
    && curl -s -L https://nvidia.github.io/nvidia-
docker/$distribution/nvidia-docker.list | sudo tee
/etc/apt/sources.list.d/nvidia-docker.list
sudo apt-get update
sudo apt-get install nvidia-container-toolkit
```

3. Restart Docker

```
sudo systemctl restart docker
```

4. Verify by running the command:

```
docker run -it --rm --gpus=all ubuntu nvidia-smi
```

You should see the following:

```
+-----------------------------------------------------------------------
----------+
| NVIDIA-SMI 525.125.06      Driver Version: 525.125.06    CUDA
Version: 12.0 |
|-------------------------------+----------------------+------------
----------+
```

Processor API container

Ensure you have Docker Engine installed and that you're logged in with the Docker user we've given access to in our repo. Then, run the following to download the image.

```
docker pull containers.paravision.ai/processor/processor:<tag>
```

Ports

Expose port 50051 on your machine to communicate with the API container externally (or another port if you choose to map differently).

Run Container

Run the following to start the api with appropriate port mapping.

OpenVINO (CPU)

```
docker run -dt --rm --name processor-api -p 50051:50051
containers.paravision.ai/processor/processor:<tag>
```

TensorRT (GPU)

```
docker run -dt --rm --name processor-api --gpus=all -p 50051:50051
containers.paravision.ai/processor/processor:<tag>
```

Note:

When running on GPU, the recognition models are compiled specifically for your hardware on each startup of the container. Consider adding the `-v <HOST_DIR>:<PV_TRT_ENGINE_PATH>` volume specification to the Docker run command above, to cache the output in persistent directory on the host. For more on `PV_TRT_ENGINE_PATH`, see the section below.

For recognition (precision or larger), age, deepfake, and liveness models, the following warning message will appear (expected for larger models). This message is expected and will not impact functionality.

`[libprotobuf WARNING google/protobuf/io/coded_stream.cc:604] Reading dangerously large protocol message. If the message turns out to be larger than 2147483647 bytes, parsing will be halted for security reasons.`

If you wish to enable the optional HTTP REST interface, see instructions for HTTP usage.

Notes:

- For minimum and recommended system requirements, see System requirements.

> ⚠ The compiled models don't automatically update when recognition models change. Make sure to clear out the cache directory when switching between processor container versions.

Starting in version 7.2.0, you may also run the docker container as the non-root, `paravision` user. To do so, use the `-u paravision` argument in your `docker run` command.

Options

Prior to 6.0.0 options did not include the `PV_` prefix.

The following options can be configured via environment variables when running the container. For example, `-e PV_SSL=on`.

PV_DETECTION_MODEL

Detection models:

- `default` is optimized for 1:1 image ratio
- `streaming` for 16:9
- `mobile` for liveness.
- `gen6.1` is an improved model for 1:1 image ratio (experimental)

Default: `default`.

Please specify `mobile` detection model for user requests from mobile or edge devices.

PV_GRPC_INTERFACE

When set to `on` enabled the GRPC Interface. Default `on`.

PV_GRPC_BIND_ADDRESS

Set the bind address for the GRPC Interface. Default `0.0.0.0:50051`

PV_HTTP_INTERFACE

When set to `on`, enables HTTP Interface. Default `off`

PV_HTTP_BIND_ADDRESS

Set the bind address for the HTTP Interface. Default `0.0.0.0:8081`

PV_HTTP_AUTH

When set to `on`, enables HTTP basic authentication in HTTP Interface. Default: `off`.

PV_HTTP_AUTH_USER

Username for HTTP basic authentication. Default: *empty*.

PV_HTTP_AUTH_PASSWORD

Password for HTTP basic authentication. Default: *empty*.

PV_HTTP_AUTH_PASSWORD_FILE

Path inside the container to a file with password for HTTP basic authentication. Can be used instead of `PV_HTTP_AUTH_PASSWORD`. Default: *empty*.

PV_INFERENCE_WORKERS

The number of workers processing requests. Increase the default if you require a higher throughput. If you are using TensorRT you can also specify a comma separated list of workers per GPU index where the GPU index is the position in the list, for example `1,0,1,2` with this configuration GPU-0 has 1 worker, GPU-1 has 0 workers, GPU-2 has 1 worker and GPU-3 has 2 workers, you can get the GPU index ids from `nvidia-smi`. Default: `1`.

PV_SCORING_MODE

Scoring mode affects recognition and determines how faces are compared to each other. The legacy way of comparison is based on Euclidean distances of embeddings and is selected by setting the value as `standard`. The new and more accurate method of comparison takes into account the quality of the detected face, and is selected by setting the value as `enhanced`. In `enhanced` mode produced embeddings also store additional dimension that represents quality of the face. Default: `enhanced`.

PV_SSL

When set to `on`, enables SSL/TLS encryption for gRPC (and also HTTP if interface is enabled). Requires `PV_SSL_CERT` and `PV_SSL_KEY` files to exist within the container. You can mount them from the host for example with `-v HOST_PATH:/server.crt -v HOST_PATH:/server.key` volume specification in `docker` command line. Only connections using TLS 1.2 or higher are supported. Default: `off`.

PV_SSL_CERT

Path inside the container to the SSL Certificate. Default: `/server.crt`.

PV_SSL_KEY

Path inside the container to the SSL Private Key. Default: `/server.key`.

Refer to the tutorial ↗ for more details on SSL set-up.

PV_OPENVINO_THREADS_LIMIT

The amount of CPUs that OpenVINO will try to use. If not set, OpenVINO will try to use all the CPUs available in the container. Default: *empty*. This is tht total number of threads across all workers. So if you have 5 workers and limit the OpenVINO runtime to 5 threads, each worker will get 1 thread.

PV_CONNECTION_AGE_MS

Maximum lifespan of one connection to the server, in milliseconds. Accepted values range from `1` to `2^31 - 1`, where the latter represents infinity. There is a 10 seconds grace period that allows already started requests to complete. Our clients will automatically re-connect to the server when the current connection gets closed, so using this setting does not require any changes client-side. This setting can be useful when using client-side load-balancing. Default: `2^31 - 1`.

PV_PROMETHEUS_METRICS

When set to `on`, enables Prometheus ↗ metrics in the Processor container. Default: `off`.

Metrics will be exposed on the port `8091`. The exported metrics contain a Histogram ↗ named `pv_requests_latency_ms` this metric has multiple labels so you can measure the latency per mdoel infer request. The labels are `protocol` either `GRPC` or `HTTP`, `method` one of `get_faces`, `get_embedding_from_landmarks`, `get_face_from_prepared`, or `get_attributes` , finally we have a label for the `success` either `true` or `false` to indicate if the job was successful.

Refer to the tutorial ↗ for more details on Prometheus metrics.

PV_PROMETHEUS_BIND_ADDRESS

Sets the bind address for the Prometheus metric scrape endpoint. Default `0.0.0.0:8091`

PV_MODEL_PATH

The path to the models. Default *`unset`*
Unless you are using one of our containers, if so the default path is already set for you. To either `/gpu_models` or `/cpu_models`

PV_LOG_LEVEL

Change the log level of the processor. Default `info`

PV_LOG_FORMAT

Customize the log format using a pattern string as described here ↗.

PV_ATTRIBUTE_SDK

Enable the Attribute SDK, this will give access to gender / mask / smile / headwear / glasses / age detecion. Default `on`

PV_TRT_ENGINE_PATH

Allows users to specify a custom cache directory for TensorRT engine files. The TensorRT accelerator takes a long time to start up. By specifying a cache path and then volume mounting a directory into the container will drastically reduce the initial container start time. However you have to be careful with how the engine caches work together. Different GPUs (even if they are the same model) shouldn't share the same cache, as it can result in performance loss. If you are deploying in K8S you should not use a PVC (Persistent Volume Claim) as these can be mapped onto any node and shared between nodes. You should rather opt to use a host volume mount to store the engine files on the host machine. If you are running multiple instances of processor on the same machine, with the same settings you can share a single cache, if you are using different values you should use a different cache. The first

inference request after a successful cache load will be slower than normal since the cache will be cold, but after the first requests performance should be the same as if you serialized the engine from scratch.

PV_DOC_AUTH_URI

Integration with Regula ↗ for identity document authentication. Assumes prior setup ↗ and licensing of Regula docker container, see docker `compose.yaml` file below for sample configuration.

> Both HTTP & HTTPS endpoints are supported.
>
> For example: `PV_DOC_AUTH_URI=https://api.regulaforensics.com:443` will configure processor to utilize the regula demo endpoint (note this endpoint has limited functionality as it is only intended to be used for demo purposes).

```
version: '3.8'

services:
  processor-api:
    image: containers.paravision.ai/processor/processor:<tag>
    container_name: processor-api
    environment:
      - PV_DOC_AUTH_URI=http://regula:8080
    ports:
      - "50051:50051"
    networks:
      - pvnetwork
    restart: always

  regula:
    image: regulaforensics/docreader:7.5
    container_name: regula
    networks:
      - pvnetwork
    restart: always
    volumes:
      - ./regula.license:/app/extBin/unix/regula.license

networks:
  pvnetwork:
    driver: bridge
```

PV_USAGE_DB_URI

> Starting in v7.9.0, Processor API can be started with Clickhouse to count feature transactions for the purposes of usage reporting. This feature is currently in beta.

This environment variable specifies the connection URI path to the Clickhouse database container and enables the ability to count feature transactions (e.g. Liveness, Age, Deepfake etc.) and generate reports on demand. features are counted automatically when a request to endpoints such as `process_full_image` or `process_aligned_image` are successful. Reports can be generated by calling the `get_usage_report` endpoint (available in both gRPC and HTTP) and sent to Paravision.

ProcessAPI and the Clickhouse docker container can be started standalone or using orchestration tools like Kubernetes and using Docker compose like in the sample below. Please ensure the data stored by the Clickhouse database container is persisted by volume mounting or other means that are appropriate for your scenario.

For more information on Clickhouse, please see here ↗.

```
version: '3.8'

services:
  processor-api:
    image: containers.paravision.ai/processor/processor:<tag>
    container_name: processor-api
    # The PV_USAGE_DB_URI must match that environment variables passed
to the Clickhouse database container
    environment:
      -
PV_USAGE_DB_URI=db://paravision:password@clickhouse:9000/paravision_usa
ge
    ports:
      - "50051:50051"
    networks:
      - pvnetwork
    restart: always

  clickhouse:
    container_name: clickhouse
    image: clickhouse/clickhouse-server
    # Any variables can be set for the Clickhouse database container
    environment:
      - CLICKHOUSE_DB=paravision_usage
      - CLICKHOUSE_USER=paravision
      - CLICKHOUSE_DEFAULT_ACCESS_MANAGEMENT=1
      - CLICKHOUSE_PASSWORD=password
    # It is strongly advised to volume mount the following paths of the
Clickhouse database container for data persistence.
    volumes:
      - './storage/clickhouse:/var/lib/clickhouse/'
      - './storage/clickhouse-logs:/var/log/clickhouse-server/'
    networks:
      - pvnetwork
    restart: always

networks:
  pvnetwork:
    driver: bridge
```

PV_INJECTION_DETECTION

ⓘ Starting in v7.9.0, Processor API can be started with Injection Attack Detection (IAD). This feature requires pairing with a Paravision capture client to successfully utilize

injection detection processing. See Injection Detection (Experimental) for more details.

Enable or disable injection detection support, and its associated resource usage. Default: `off`

Client

C#

1. Add ProcessorCSharpClient nuget source

```
dotnet nuget add source https://nuget.fury.io/<token>/<username>/
```

where `<token>` and `<username>` are your credentials to the Gemfury repository we provide you.

2. Add ProcessorCSharpClient nuget package to your project. Make sure you are in the directory that has your csproj file

```
dotnet add Application.csproj package ProcessorCSharpClient -v 7.0.0
```

where `Application.csproj` is the csproj file for your application.

C++

1. Download pre-built libraries for your system

- Repository: `https://<user>:<password>@paravision.mycloudrepo.io/repositories/processor-cpp-client/`
- Linux 64-bit: `paravision-processor-client-v7.0.0-linux-x64.tar.gz`
- Windows 64-bit, MSVC v140+ (Visual Studio 2015+): `paravision-processor-client-v7.0.0-windows-x64-msvc-v14x.zip`

Where `<user>` and `<password>` are your credentials to the repository we provide you.

2. Unpack library files (`lib/`, and `bin/` on Windows) and headers(`include/`) to desired directory and setup linking in your project

Go

1. Configure Go to use Gemfury proxy.

```
export GOPROXY=https://<token>@go-proxy.fury.io/<username>/
export GOPRIVATE="golang.paravision.ai/*"
export GONOPROXY=none
```

where `<token>` and `<username>` are your credentials to the Gemfury repository we provide you.

2. Add our client package to the list of dependencies in the `go.mod` of your project:

```
require golang.paravision.ai/processor-go-client/v7 v7.0.0
```

3. Run `go mod download` in your Go project.

Java

1. Configure your dependency manager to use the Maven repository at `https://maven.fury.io/paravision`, with HTTP basic auth, using the repository token we give you as username, and `NOPASS` as the password.

For example, with Gradle you can use:

```
repositories {
    jcenter()  // Not required, but our repository does not mirror
               // any packages, so you'll likely need some other
               // base repository.

    maven {
        name = 'paravision'
        url = 'https://maven.fury.io/paravision'

        authentication {
            basic(BasicAuthentication)
        }

        credentials {
            username = "<paravision_token_here>"
            password = "NOPASS"
        }
    }
}
```

2. Add the `processorclient:7.0.0` dependency from `ai.paravision` group to your project.

A fat JAR is also available. It bundles all transitive dependencies together with Paravision library. To use it, append `.fat` to version number, for instance: `processorclient:7.0.0.fat`.

Continuing the Gradle example:

```
dependencies {
    implementation 'ai.paravision:processorclient:7.0.0'
}
```

3. The resources provided by Processor Java client are available in the `ai.paravision.processor` package. Thus your Java source files might use:

```
import ai.paravision.processor.*;
```

Node

1. Add our client package to the list of dependencies in the `package.json` of your project:

```
"@paravision/processor-js-client": "^7.0.0"
```

2. Add the following to your `.npmrc` file:

```
@paravision:registry=https://npm.fury.io/<username>/
registry=https://registry.npmjs.org/

//npm.fury.io/<username>/:_authToken=<token>
```

where `<token>` and `<username>` are your credentials to the Gemfury repository we provide you.

3. Run `npm install` in your node project.

Python

1. Install Processor client package

```
pip3 install \
    --no-cache-dir \
    --extra-index-url https://<user>:
<pass>@paravision.mycloudrepo.io/repositories/processor-python-
client \
    paravision-processor-client==7.0.0
```

where `<user>` and `<password>` are your credentials to the repository we provide you.

Ruby

1. Install Processor client package `paravision-processor-client` from the `https://gem.fury.io/paravision/` source, authorizing with the credentials to the Gemfury repository we provide you.

RubyGems

Simply install the `paravision-processor-client` gem

```
gem install paravision-processor-client -v 7.0.0 --source
https://<paravision_token_here>@gem.fury.io/paravision/
```

Bundler 1.8+

Add `paravision-processor-client` gem to your `Gemfile`

```
gem 'paravision-processor-client', '7.0.0', source:
'https://gem.fury.io/paravision/'
```

and configure Bundler to use your repository token

```
bundle config https://gem.fury.io/paravision/
<paravision_token_here>
```

Finally, install the client package

```
bundle install
```

Bundler pre-1.8

Add `paravision-processor-client` gem to your `Gemfile`

```
gem 'paravision-processor-client', '7.0.0', source: 'https://#
{ENV['FURY_AUTH']}@gem.fury.io/paravision/'
```

and define `FURY_AUTH` environment variable before using Bundler

```
export FURY_AUTH=<paravision_token_here>
```

or using `bundler config` to globally specify the auth info

```
bundle config https://gem.fury.io/paravision/
<paravision_token_here>
```

However it is possible for Bundler to occasionally put your token into Gemfile.lock. It is safe to remove the token and commit just the URL without

credentials into your SCM

```
GEM
  remote: https://gem.fury.io/paravision/
  ...
```

Finally, install the client package

```
bundle install
```

System Requirements

The below recommendations are based on tests with 94KB jpg image running against a single Processor v7.6.0 instance.

> The system requirements are highly dependent on your usage patterns, you should run tests with your representative images and use cases to figure out the optimal requirements. The ones given below should be a reasonable starting point though.

Minimum requirements should allow you to run a very simple (and slow) development machine for evaluation purposes while recommended ones should be more reasonable average production requirements.

Machine specification

Basic

Minimum requirements

model / engine	OpenVINO	TensorRT
gen6-fast	1 vCPU, 1.4 GB RAM	1 vCPU, 3.3 GB RAM, 0.5 GB VRAM
gen6-balanced	1 vCPU, 1.9 GB RAM	1 vCPU, 3.7 GB RAM, 0.8 GB VRAM
gen6-precision	1 vCPU, 3.3 GB RAM	1 vCPU, 3.8 GB RAM, 1.5 GB VRAM
gen6-precision-plus	1 vCPU, 5.2 GB RAM	1 vCPU, 5.1 GB RAM, 2.7 GB VRAM

Recommended requirements

model / engine	OpenVINO	TensorRT

gen6-fast	16 vCPU, 2.0 GB RAM	16 vCPU, 4.5 GB RAM, 0.5 GB VRAM
gen6-balanced	16 vCPU, 2.7 GB RAM	16 vCPU, 4.9 GB RAM, 0.8 GB VRAM
gen6-precision	24 vCPU, 4.6 GB RAM	16 vCPU, 5.1 GB RAM, 1.5 GB VRAM
gen6-precision-plus	24 vCPU, 7.1 GB RAM	16 vCPU, 6.9 GB RAM, 2.7 GB VRAM

Age & Liveness

Minimum requirements

model / engine	OpenVINO	TensorRT
gen6-fast	1 vCPU, 6.3 GB RAM	1 vCPU, 13.3 GB RAM, 4.8 GB VRAM
gen6-balanced	1 vCPU, 6.9 GB RAM	1 vCPU, 13.1 GB RAM, 5.1 GB VRAM
gen6-precision	1 vCPU, 8.5 GB RAM	1 vCPU, 13.5 GB RAM, 5.8 GB VRAM
gen6-precision-plus	1 vCPU, 10.5 GB RAM	1 vCPU, 14.8 GB RAM, 7.0 GB VRAM

Recommended requirements

model / engine	OpenVINO	TensorRT
gen6-fast	16 vCPU, 8.9 GB RAM	16 vCPU, 17.8 GB RAM, 4.8 GB VRAM
gen6-balanced	16 vCPU, 9.6 GB RAM	16 vCPU, 17.5 GB RAM, 5.1 GB VRAM
gen6-precision	24 vCPU, 11.8 GB RAM	16 vCPU, 18.0 GB RAM, 5.8 GB VRAM

gen6-precision-plus	24 vCPU, 14.5 GB RAM	16 vCPU, 19.7 GB RAM, 7.0 GB VRAM

Deepfake

Minimum requirements

model / engine	OpenVINO	TensorRT
gen6-fast	1 vCPU, 6.4 GB RAM	1 vCPU, 5.2 GB RAM, 5.0 GB VRAM
gen6-balanced	1 vCPU, 9.5 GB RAM	1 vCPU, 4.8 GB RAM, 5.3 GB VRAM
gen6-precision	1 vCPU, 8.3 GB RAM	1 vCPU, 5.1 GB RAM, 6.0 GB VRAM
gen6-precision-plus	1 vCPU, 10.6 GB RAM	1 vCPU, 5.0 GB RAM, 7.3 GB VRAM

Recommended requirements

model / engine	OpenVINO	TensorRT
gen6-fast	24 vCPU, 8.6 GB RAM	16 vCPU, 6.9 GB RAM, 5.0 GB VRAM
gen6-balanced	24 vCPU, 6.3 GB RAM	16 vCPU, 13.3 GB RAM, 5.3 GB VRAM
gen6-precision	24 vCPU, 11.3 GB RAM	16 vCPU, 6.8 GB RAM, 6.0 GB VRAM
gen6-precision-plus	24 vCPU, 14.3 GB RAM	16 vCPU, 6.7 GB RAM, 7.3 GB VRAM

All

Minimum requirements

model / engine	OpenVINO	TensorRT
gen6-fast	1 vCPU, 11.4 GB RAM	1 vCPU, 15.4 GB RAM, 9.3 GB VRAM
gen6-balanced	1 vCPU, 12.1 GB RAM	1 vCPU, 14.7 GB RAM, 9.6 GB VRAM
gen6-precision	1 vCPU, 13.5GB RAM	1 vCPU, 13.8 GB RAM, 10.3 GB VRAM
gen6-precision-plus	1 vCPU, 15.9 GB RAM	1 vCPU, 15.3 GB RAM, 11.5 GB VRAM

Recommended requirements

model / engine	OpenVINO	TensorRT
gen6-fast	24 vCPU, 15.6 GB RAM	16 vCPU, 20.6 GB RAM, 9.3 GB VRAM
gen6-balanced	24 vCPU, 16.6 GB RAM	16 vCPU, 19.6 GB RAM, 9.6 GB VRAM
gen6-precision	24 vCPU, 18.6 GB RAM	16 vCPU, 18.4 GB RAM, 10.3 GB VRAM
gen6-precision-plus	24 vCPU, 21.8 GB RAM	16 vCPU, 20.4 GB RAM, 11.5 GB VRAM

To run our containers with GPU support on Windows, you need to use Windows 10, version 1903 (or higher) or Windows 11, with the *most recent Linux kernel in WSL2* (it can be updated with `wsl --update`). For instructions on the setup, follow our installation guide.

Software requirements

- Operating System: Ubuntu 22.04 Or RHEL 7.6 or above
- CUDA 12.1

Hardware reference machine specification

All of the above specs were tested on the following hardware.

- CPU
  - EC2 c5.12xlarge
  - RAM: 96 GB
- GPU
  - 1 Nvidia T4
  - GCP n1-standard-16 with skylake
  - RAM: 60GB

Usage

The Processor API container runs a gRPC server that allows you to get bounding boxes, landmarks, embeddings and other face information from your images. The guide below will show you how to use our various clients to interact with that server.

Initialization

Create a `Client` object.

C#

```
Client Client(string host, bool ssl = false, string sslCert = null,
string lbPolicy = null)
```

C++

```
Client Client(string host, bool ssl = false, string ssl_cert = "",
string lb_policy = "")
```

Go

```
NewClient(ctx context.Context, host string, ssl bool, sslCert
[]byte, lbPolicy string)(*Client, error)
```

Java

```
Client Client(String host, boolean ssl = false, byte[] sslCert =
null, String lbPolicy = null)
```

Node

```
Client(host: string, ssl: Boolean = false, sslCert: string = null,
lbPolicy: string = null) => Client
```

Python

```
Client(
    host: str,
    ssl: bool = False,
    ssl_cert: Optional[bytes] = None,
    lb_policy: Optional[str] = None
) -> Client
```

Async client is also available since v6.3.0:

```
AsyncClient(
    host: str,
    ssl: bool = False,
    ssl_cert: Optional[bytes] = None,
    lb_policy: Optional[str] = None
) -> AsyncClient
```

Ruby

```
Client.new(host, ssl: false, ssl_cert: nil, lb_policy: nil) ->
Client
```

Arguments

ctx *(Go only)*: `Context` ↗ to be used with underlying gRPC calls.

host: A string, the url where the Processor API is running. For example, if you are running the API in a local Docker container on port `50051` with port forwarding, you should use a host of `localhost:50051`.

ssl: A boolean, describes whether to enable SSL/TLS encryption (requires encryption enabled on the server).

ssl_cert: A string, certificate to use for validation instead of built-in OS Root Certificates.

lb_policy: A string, name of the policy to be used by gRPC in the client side load balancing.

Return

client: The Processor `Client` object.

Example

C#

```
using Processor;

// Insecure connection
var client = new Processor.Client("localhost:50051");

// With SSL and custom certificate
var sslCert = File.ReadAllText("ssl.cert");
var client = new Processor.Client("localhost:50051", true, sslCert);

// With round_robin client side load balancing
var client = new Processor.Client("dns:///localhost:50051", false,
null, "round_robin");
```

C++

```cpp
#include <paravision/processor.h>

// Insecure connection
paravision::processor::Client client("localhost:50051");

// With SSL and custom certificate
string ssl_cert;
{
    auto ss = ostringstream{};
    ifstream file("ssl.cert", ifstream::in | ifstream::binary);
    ss << file.rdbuf();
    ssl_cert = ss.str();
}

paravision::processor::Client client("localhost:50051", true,
ssl_cert);

// With round_robin client side load balancing
paravision::processor::Client client("dns:///localhost:50051",
false, "", "round_robin");
```

Go

```
import {
    "io/ioutil"

    processor "golang.paravision.ai/processor-go-client/v6"
    pb "golang.paravision.ai/processor-go-client/v6/proto"
}

ctx, cancel := context.WithCancel(context.Background())
defer cancel()

// Insecure connection
client, err := processor.NewClient(ctx, "localhost:50051", false,
nil, "")

// With SSL and custom certificate
sslCert, err := ioutil.ReadFile("ssl.cert")
if err != nil {
    return nil, err
}

client, err := processor.NewClient(ctx, "localhost:50051", true,
sslCert, "")

// With round_robin client side load balancing
client, err := processor.NewClient(ctx, "dns:///localhost:50051",
false, nil, "round_robin")
```

Java

```
import ai.paravision.processor.*;

// Insecure connection
var client = new Client("localhost:50051");

// With SSL and custom certificate
var sslCert = Files.readAllBytes("ssl.cert")
var client = new Client("localhost:50051", true, sslCert);

// With round_robin client side load balancing
var client = new Client("dns:///localhost:50051", false, null,
"round_robin");
```

Node

```
const { Client, ProcessFullImageOptions,
ProcessAlignedFaceImageOptions } = require('@paravision/processor-
js-client')

// Insecure connection
const client = new Client("localhost:50051");

// With SSL and custom certificate
const sslCert = fs.readFileSync("ssl.cert");
const client = new Client("localhost:50051", true, sslCert);

// With round_robin client side load balancing
const client = new Client("dns:///localhost:50051", false, null,
"round_robin");
```

Python

```
import paravision_processor as processor

# Insecure connection
client = processor.Client("localhost:50051")

# With SSL and custom certificate
with open("ssl.cert", "rb") as certfile:
    ssl_cert = certfile.read()

client = processor.Client("localhost:50051", ssl=True,
ssl_cert=ssl_cert)

# With round_robin client side load balancing
client = processor.Client("dns:///localhost:50051",
lb_policy="round_robin")
```

Ruby

```
require 'paravision_processor/client'
include Paravision

# Insecure connection
client = Processor::Client.new('localhost:50051')

# With SSL and custom certificate
ssl_cert = File.binread('ssl.cert')
client = Processor::Client.new('localhost:50051', ssl: true,
ssl_cert: ssl_cert)

# With round_robin client side load balancing
client = Processor::Client.new('dns:///localhost:50051', lb_policy:
"round_robin")
```

Refer to the tutorial ↗ for more details on SSL connection set-up.

Note: In some implementations before you can start making requests, you must first use the `Connect` function below to connect to the server.

All methods that perform gRPC calls underneath provide a way to set a timeout (either explicitly, or through context like in Golang). It is considered a good practice ↗ to set a timeout.

In some languages *(C#, Node.js, Ruby)* the system clock adjustments during gRPC calls with a timeout can cause unexpected behaviour.

gRPC supports client-side load balancing. To activate it in our clients, you need to:

- provide a correct value for the `lb_policy` constructor argument. The full list of policies supported by gRPC is available in its load balancing documentation ↗.
- provide the host address as a full URI with dns scheme (`dns:///<dns resolvable name>`). This is because internally gRPC is using DNS to resolve the list of available backend addresses.

You can also set the `PV_CONNECTION_AGE_MS` environment variable on the server to make your clients reconnect to the server periodically, allowing them to re-resolve

backend addresses. This is useful when your server deployment scales up, as clients will be aware of the changes faster and will be able to accommodate the changes into load balancing strategy sooner.

Connect

Connect connects your client to the Processor API's gRPC server.

C#

```
void Connect(float? timeout = null)
```

C++

```
paravision::Status Connect(float timeout = -1)
```

Java

```
void connect(float timeout = -1)
```

Node

```
connect(float timeout = null) => Promise<void>
```

Python

```
connect(timeout: Optional[float] = None) -> None
```

Ruby

```
connect(timeout: nil) -> nil
```

Arguments

timeout: A float value, specifying the timeout in seconds for the underlying gRPC calls. No timeout is specified by a null value (where the language allows), or by -1 in the languages where float is not nullable.

Example

C#

```
client.Connect(10.23);
```

C++

```
auto status = client.Connect(10.23);
if(!status.ok())
{
    cerr << status.error_message << ": "
         << status.error_details << endl;
    exit(1);
}
```

Java

```
client.connect(10.23);
```

Node

```
client.connect(10.23)
  .then(() => {
    // use client for image processing
  })
  .catch(err => {
    console.error(err);
  })
```

Python

```
client.connect(10.23)
```

Ruby

```
client.connect(timeout: 10.23)
```

Close

Close the connection to the Processor API's gRPC server.

C#

```
void Close()
```

C++

```
void Close()
```

Go

```
Close()
```

Java

```
void close()
```

Node

```
close() => void
```

Python

```
close() -> None
```

Ruby

```
close() -> nil
```

Example

C#

```
client.Close();
```

C++

```
client.Close();
```

Go

```
client.Close()
```

Java

```
client.close();
```

Node

```
client.close()
```

Python

```
client.close()
```

Ruby

```
client.close
```

Process Full Image

Process full image takes an uncropped image and returns information about the faces in the image.

C#

```
ProcessFullImageResponse ProcessFullImage(
    byte[] image,
    IEnumerable<ProcessFullImageOptions> options,
    bool findMostProminentFace,
    ProcessFullImageLivenessValidnessParamaters
livenessValidnessParameters,
    ProcessFullImageAgeValidnessParamaters
agesV2ValidnessParameters,
    ProcessFullImageDeepfakeValidnessParamaters
deepfakeValidnessParameters,
    ImageSource imageSource = ImageSource.Unknown,
    float? timeout = null)
```

Async analogue is also available since v6.0.1:

```
async Task<ProcessFullImageResponse> ProcessFullImageAsync(
    byte[] image,
    IEnumerable<ProcessFullImageOptions> options,
    bool findMostProminentFace,
    ProcessFullImageLivenessValidnessParamaters
livenessValidnessParameters,
    ProcessFullImageAgeValidnessParamaters
agesV2ValidnessParameters,
    ProcessFullImageDeepfakeValidnessParamaters
deepfakeValidnessParameters,
    ImageSource imageSource,
    float? timeout = null)
```

C++

```
paravision::Status ProcessFullImage(
    ProcessFullImageResponse &response,
    const string &image,
    const vector<ProcessFullImageOptions> &options,
    bool find_most_prominent_face,
    const LivenessValidnessParameters &livenessValidnessParameters,
    const AgeValidnessParameters &agesV2ValidnessParameters,
    const DeepfakeValidnessParameters
&deepfake_validness_parameters,
    const ImageSource &image_source = ImageSource::UNKNOWN,
    float timeout = -1)
```

Go

```
ProcessFullImage(
    ctx context.Context,
    image []byte,
    options []pb.ProcessFullImageRequest_Options,
    findMostProminentFace bool,
    livenessValidnessParameters
*pb.ProcessFullImageRequest_LivenessValidnessParameters),
    ageV2ValidnessParameters
*pb.ProcessFullImageRequest_AgeValidnessParameters,
    deepfakeValidnessParameters
*pb.ProcessFullImageRequest_DeepfakeValidnessParameters,
    imageSource pb.ImageSource)
    (*pb.ProcessFullImageResponse, error)
```

Java

```
ProcessFullImageResponse processFullImage(
    byte[] image,
    Iterable<ProcessFullImageRequest.Options> options,
    boolean findMostProminentFace,
    ProcessFullImageRequest.LivenessValidnessParameters
livenessValidnessParameters,
    ProcessFullImageRequest.AgeValidnessParameters
agesV2ValidnessParameters,
    ProcessFullImageRequest.DeepfakeValidnessParameters
deepfakeValidnessParameters,
    ImageSource imageSource,
    float timeout
) throws ClientException
```

Node

```
processFullImage(
    image: (string|Uint8Array),
    options: Array<!ProcessFullImageOptions>,
    find_most_prominent_face: Boolean,
    liveness_validness_parameters:
ProcessFullImageLivenessValidnessParameters,
    ages_v2_validness_parameters:
ProcessFullImageAgeValidnessParameters,
    deepfake_validness_parameters:
ProcessFullImageDeepfakeValidnessParameters,
    image_source: ImageSource = ImageSource.UNKNOWN
    timeout: number = null)
    => Promise<ProcessFullImageResponse>
```

Python

```
process_full_image(
    image: bytes,
    options: Iterable[ProcessFullImageOptions],
    find_most_prominent_face: bool,
    liveness_validness_parameters:
ProcessFullImageLivenessValidnessParameters,
    ages_v2_validness_parameters:
ProcessFullImageAgeValidnessParameters,
    deepfake_validness_parameters:
ProcessFullImageDeepfakeValidnessParameters,
    image_source: ImageSource = ImageSource.UNKNOWN,
    timeout: Optional[float] = None
) -> ProcessFullImageResponse
```

Ruby

```
process_full_image(image,
    options,
    find_most_prominent_face,
    liveness_validness_parameters,
    ages_v2_validness_parameters,
    deepfake_validness_parameters,
    image_source: ImageSource::UNKNOWN,
    timeout: nil) -> ProcessFullImageResponse
```

Arguments

ctx *(Go only)*: `Context` ↗ to be used with underlying gRPC calls.

image: A bytes object, the image to process.

options: A list of `ProcessFullImageOptions`, the attributes to be included in the returned `Face` objects.

find_most_prominent_face: A boolean value, describes whether `most_prominent_face_idx` should be returned.

liveness_validness_parameters: A `ProcessFullImageLivenessValidnessParameters` struct used for liveness validation.

ages_v2_validness_parameters: A `ProcessFullImageAgeValidnessParameters` struct used for ages_v2 estimation.

deepfake_validness_parameters: A `ProcessFullImageDeepfakeValidnessParameters` struct used for deepfake detection.

image_source: A ImageSource enum used to specify the source of image (`unknown`, `mobile`, or `webcam`). Default is `unknown`.

timeout *(not available in Go)*: A float value, specifying the timeout in seconds for the underlying gRPC calls. No timeout is specified by a null value (where the language allows), or by -1 in the languages where float is not nullable.

Return

response: A `ProcessFullImageResponse` instance containing a list of `Faces` found in the image, each with the requested attributes attached, and an optional `most_prominent_face_idx`, if requested using the `find_most_prominent_face` parameter (if not -1 is returned).

status: C++, `Status` object indicating call status. If no errors occured, `status.ok()` will be `true`.

From v7.7.0 onward, Face objects from detection are now sorted based on prominence (largest face by area). Therefore, the most prominent face is always at index 0.

Note: In order to use the `LIVENESS` and `LIVENESS_VALIDNESS` outputs you must be running the liveness enabled processor container. If you request liveness from a container that does not have liveness enabled the request will fail.

Note: In order to use the `AGES_V2` and `AGES_V2_VALIDNESS` outputs you must be running the ages_v2 enabled processor container. If you request ages_v2 from a container that does not have ages_v2 enabled the request will fail.

Note: In order to use the `DEEPFAKE` and `DEEPFAKE_VALIDNESS` outputs you must be running the deepfake enabled processor container. If you request deepfake from a container that does not have deepfake enabled the request will fail.

Example

C#

```
// Get aligned face images from the source image along with the
match quality
byte[] image = File.ReadAllBytes(image_path);

var response = client.ProcessFullImage(
    image, new[] {
        pb.ProcessFullImageRequest.Types.Options.AlignedFaceImage,
        pb.ProcessFullImageRequest.Types.Options.Quality},
    false,
    null,
    null,
    null,
    pb.ImageSource.Unknown,
);

List<pb.Face> faces = response.Faces;
```

C++

```
// Get aligned face images from the source image along with the
match quality
string image;
{
    auto ss = ostringstream{};
    ifstream file(image_path, ifstream::in | ifstream::binary);
    if(!file)
    {
        cerr << "Couldn't open " << image_path << " Please make sure
it exists." << endl;
        exit(1);
    }

    ss << file.rdbuf();
    image = ss.str();
}

ProcessFullImageResponse response;
status = client.ProcessFullImage(
    response,
    image,
    {ProcessFullImageRequest::ALIGNED_FACE_IMAGE,
        ProcessFullImageRequest::QUALITY},
    false, nullptr, nullptr, nullptr, ImageSource::UNKNOWN);

if(!status.ok())
{
    cerr << "Processing image failed! " << status.error_message
         << ": " << status.error_details << endl;
    exit(1);
}

vector<Face> faces = response.faces;
```

Go

```
// Get aligned face images from the source image along with the
match quality
image, err := ioutil.ReadFile(*imagePath)
if err != nil {
    log.Fatalf("Couldn't read input image: %v", err)
}

options := []pb.ProcessFullImageRequest_Options{
    pb.ProcessFullImageRequest_ALIGNED_FACE_IMAGE,
    pb.ProcessFullImageRequest_QUALITY,
}

ctx, cancel := context.WithCancel(context.Background())
defer cancel()

response, err := client.ProcessFullImage(ctx, image, options, false,
nil, nil, nil, pb.ImageSource_UNKNOWN)
if err != nil {
    log.Fatalf("Error processing full image: %v", err)
}

faces := response.GetFaces();
```

Java

```
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.EnumSet;

// Get aligned face images from the source image along with the
match quality
byte[] image = Files.readAllBytes(Paths.get(imagePath));
ProcessFullImageResponse response = client.processFullImage(
    image,
     EnumSet.of(
        ProcessFullImageRequest.Options.ALIGNED_FACE_IMAGE,
        ProcessFullImageRequest.Options.QUALITY),
    true,

ProcessFullImageRequest.LivenessValidnessParameters.newBuilder().bui
ld(),

ProcessFullImageRequest.AgeValidnessParameters.newBuilder().build(),

ProcessFullImageRequest.DeepfakeValidnessParameters.newBuilder().bui
ld(),
    ImageSource.UNKNOWN,
);
List<Face> faces = response.getFacesList();
```

Node

```
const fs = require('fs');

// Get aligned face images from the source image along with the
match quality
function readImage(image_path) {
  fs.readFile(image_path, (err, image) => {
    if (err) throw err;

    let response = await client.processFullImage(
        image, [
            ProcessFullImageOptions.ALIGNED_FACE_IMAGE,
            ProcessFullImageOptions.QUALITY],
            false, null, null, null, ImageSource.UNKNOWN);

    let faces = response.getFacesList();
    for (const face of faces) {
            // use face
            // e.g. face.getQuality();
            // e.g. face.getAlignedFaceImage();
    }
  });
}
```

Python

```
# Get aligned face images from the source image along with the match
quality
with open(image_path, "rb") as image:
    image_data = image.read()
    response = client.process_full_image(
        image_data,
        [
            processor.ProcessFullImageOptions.ALIGNED_FACE_IMAGE,
            processor.ProcessFullImageOptions.QUALITY,
        ],
        False,
        None,
        None,
        None,
        Processor::ImageSource::UNKNOWN,
    )
    faces = response.faces
```

Ruby

```
# Get aligned face images from the source image along with the match
quality
image_data = File.binread(image_path)
response = client.process_full_image(
    image_data, [

Paravision::Processor::ProcessFullImageOptions::ALIGNED_FACE_IMAGE,
        Paravision::Processor::ProcessFullImageOptions::QUALITY],
false, nil, nil, nil,
        Processor::ImageSource::UNKNOWN)
faces = response.faces
```

Process Prepared Face Image

Process prepared face image takes one or two prepared images and returns information about the face.

C#

```
Face ProcessPreparedFaceImage(
    byte[] image,
    IEnumerable<ByteString> attributesImages,
    IEnumerable<ByteString> deepfakeImages,
    byte[] livenessImage,
    byte[] agesV2Image,
    IEnumerable<ProcessAlignedFaceImageOptions> options,
    float? timeout)
```

Async analogue is also available since v6.0.1:

```
async Task<Face> ProcessPreparedFaceImageAsync(
    byte[] image,
    IEnumerable<ByteString> attributesImages,
    IEnumerable<ByteString> deepfakeImages,
    byte[] livenessImage,
    byte[] agesV2Image
    IEnumerable<ProcessAlignedFaceImageOptions> options,
    float? timeout)
```

C++

```
paravision::Status ProcessPreparedFaceImage(
    Face &face,
    const string &image,
    const google::protobuf::RepeatedPtrField<std::string>
&attributes_images_data,
    const google::protobuf::RepeatedPtrField<std::string>
&deepfake_images_data,
    const string &liveness_image_data,
    const string &ages_v2_image_data,
    const vector<ProcessPreparedFaceImageOptions> &options,
    float timeout = -1)
```

Go

```
ProcessPreparedFaceImage(
    ctx context.Context,
    image []byte,
    attributesImages [][]byte,
    deepfakeImages [][]byte,
    livenessImage []byte,
    agesV2Image []byte,
    options []pb.ProcessPreparedFaceImageRequest_Options)
    (*pb.Face, error)
```

Java

```
Face processPreparedFaceImage(
    byte[] image,
    Iterable<ByteString> attributes_images,
    Iterable<ByteString> deepfake_images,
    byte[] liveness_image,
    byte[] ages_v2_image,
    Iterable<ProcessPreparedFaceImageRequest.Options> options,
    float timeout = -1
) throws ClientException
```

Node

```
processPreparedFaceImage(
    image: (string|Uint8Array),
    attributes_images: Array<string>,
    deepfake_images: Array<string>,
    liveness_image: (string|Uint8Array),
    ages_v2_image: (string|Uint8Array),
    options: Array<!ProcessPreparedFaceImageOptions>,
    timeout: number = null)
    => Promise<Face>
```

Python

```
process_prepared_face_image(
    image: bytes,
    attributes_images = "AttributesImagesLike",
    deepfake_images = "DeepfakeImagesLike",
    liveness_image: bytes,
    ages_v2_image: bytes,
    options: Iterable["ProcessPreparedFaceImageOptions.V"],
    timeout: Optional[float] = None
) -> Face
```

Ruby

```
process_prepared_face_image(image, attributes_images,
deepfake_images, liveness_image, ages_v2_image, options, timeout:
nil) -> Face
```

Arguments

ctx *(Go only)*: `Context` ↗ to be used with underlying gRPC calls.

image: A bytes object, the aligned face image to process for EMBEDDING option. This can be generated by calling process full image with the `ALIGNED_FACE_IMAGE` option, or by accessing the `recognition_input_image` image of a Face emitted by our Streaming container ↗.

attributesImages: An array of images to process for all attributes options. This can be generated by calling process full image with the `ATTRIBUTES_IMAGES` option, or by accessing the `attributes_images` images of a Face emitted by our Streaming container ↗.

deepfakeImages: An array of images to process for all deepfake options. This can be generated by calling process full image with the `DEEPFAKE_IMAGES` option.

livenessImage: A bytes object, the liveness image to process for all liveness options. This can be generated by calling process full image with the `LIVENESS_IMAGE` option.

agesV2Image: A bytes object, the ages_v2 image to process for all ages_v2 options. This can be generated by calling process full image with the `AGES_V2_IMAGE` option.

options: A list of `ProcessPreparedFaceImageOptions`, the attributes to be included in the returned `Face` object.

timeout *(not available in Go)*: A float value, specifying the timeout in seconds for the underlying gRPC calls. No timeout is specified by a null value (where the language allows), or by -1 in the languages where float is not nullable.

> The client application should employ mechanisms to ensure the correct image(s) are used for inputs: `image`, `attributesImages`, `deepfakeImages`, `livenessImage`, and `agesV2Image`. The incorrect image can sometimes lead to erroneous results.

Return

face: A `Face` with the requested attributes attached.

status: C++, `Status` object indicating call status. If no errors occured, `status.ok()` will be `true`.

Example

C#

```
// Get embedding for aligned face image
pb.Face faceData = client.ProcessPreparedFaceImage(
        face.AlignedFaceImage.ToByteArray(),
        face.AttributesImages,
        face.DeepfakeImages,
        face.livenessImage.ToByteArray(),
        face.agesV2Image.ToByteArray(),
        new []
{pb.ProcessPreparedFaceImageRequest.Types.Options.Embedding});
```

C++

```
// Get embedding for aligned face image
Face face_data;
status = client.ProcessPreparedFaceImage(
    face_data,
    face.aligned_face_image(),
    face.attributes_images(),
    face.deepfake_images(),
    face.liveness_image(),
    face.ages_v2_image(),
    {ProcessPreparedFaceImageRequest::EMBEDDING});

if(!status.ok())
{
    cerr << "Processing image failed! " << status.error_message
         << ": " << status.error_details << endl;
    exit(1);
}
```

Go

```
// Get embedding for aligned face image
options := []pb.ProcessPreparedFaceImageRequest_Options{
    pb.ProcessPreparedFaceImageRequest_EMBEDDING}

ctx, cancel := context.WithCancel(context.Background())
defer cancel()

faceData, err := client.ProcessPreparedFaceImage(ctx,
face.AlignedFaceImage, face.AttributesImages, face.DeepfakeImages,
face.LivenessImage, face.AgesV2Image, options)
if err != nil {
    log.Fatalf("Error processing aligned image: %v", err)
}
```

Java

```
// Get embedding for aligned face image
Face faceData = client.processPreparedFaceImage(
        face.getAlignedFaceImage().toByteArray(),
        face.getAttributesImagesList(),
        face.getDeepfakeImagesList(),
        face.getLivenessImage().toByteArray(),
        face.getAgesV2Image().toByteArray(),

EnumSet.of(ProcessPreparedFaceImageRequest.Options.EMBEDDING));
```

Node

```
// Get embedding for aligned face image
let aligned_face_image = face.getAlignedFaceImage();
let attributes_images = face.getAttributesImagesList();
let deepfake_images = face.getDeepfakeImagesList();
let liveness_image = face.getLivenessImage();
let ages_v2_image = face.getAgesV2Image();
client.processPreparedFaceImage(aligned_face_image,
attributes_images, deepfake_images, liveness_image, ages_v2_image [
        ProcessPreparedFaceImageOptions.EMBEDDING])
    .then(face => {
      // use face
      // e.g. face.getEmbeddingList();
    })
    .catch(err => {
      console.error(err);
    })
```

Python

```
# Get embedding for aligned face image
face_data = client.process_prepared_face_image(
    face.aligned_face_image,
    face.attributes_images,
    face.deepfake_images,
    face.liveness_image,
    face.ages_v2_image,
    [processor.ProcessPreparedFaceImageOptions.EMBEDDING]
)
```

Ruby

```ruby
# Get embedding for aligned face image
face_data = client.process_prepared_face_image(
    face.aligned_face_image,
    [face.attributes_images[0], face.attributes_images[1]],
    [face.deepfake_images[0], face.deepfake_images[1]],
    face.liveness_image, face.ages_v2_image,

[Paravision::Processor::ProcessPreparedFaceImageOptions::EMBEDDING])
```

Get Metadata

Get metadata returns the name of the recognition model and the name of the engine that your processor container is running in addition to the weight and bias being used.

C#

```csharp
pb.MetadataResponse GetMetadata(float? timeout)
```

Async analogue is also available since v6.0.1:

```csharp
async Task<pb.MetadataResponse> GetMetadataAsync(float? timeout)
```

C++

```cpp
paravision::Status GetMetadata(MetadataResponse &metadata, float
timeout = -1)
```

Go

```
GetMetadata(ctx context.Context) (*pb.MetadataReponse, error)
```

Java

```
MetadataResponse getMetadata(float timeout = -1) throws
ClientException
```

Node

```
getMetadata(timeout: number = null) => Promise<MetadataResponse>
```

Python

```
get_metadata(timeout: Optional[float] = None) -> MetadataResponse
```

Ruby

```
get_metadata(timeout: nil) -> MetadataResponse
```

Arguments

ctx *(Go only)*: `Context` ↗ to be used with underlying gRPC calls.

timeout *(not available in Go)*: A float value, specifying the timeout in seconds for the underlying gRPC calls. No timeout is specified by a null value (where the language allows), or by -1 in the languages where float is not nullable.

Return

metadata: A `MetadataResponse` object, containing the name of the model and the name of the engine your processor container is running in addition to the weight and bias.

status: C++, `Status` object indicating call status. If no errors occured, `status.ok()` will be `true`.

Example

C#

```
var metadata = client.GetMetadata();
```

C++

```
MetadataResponse metadata;
auto status = client.GetMetadata(metadata);
if(!status.ok())
{
    cerr << "Error getting metadata!" << status.error_message
         << ": " << status.error_details << endl;
    exit(1);
}
```

Go

```
ctx, cancel := context.WithCancel(context.Background())
defer cancel()

metadata, err := client.GetMetadata(ctx)
if err != nil {
    log.Fatalf("Error getting metadata, %v", err)
}
```

Java

```
var metadata = client.getMetadata();
```

Node

```
client.getMetadata()
  .then(metadata => {
    console.log(metadata);
  })
  .catch(err => {
    console.error(err);
  })
```

Python

```
metadata = client.get_metadata()
```

Ruby

```
metadata = client.get_metadata
```

Get Match Score

Get Match Score takes a pair of embeddings and returns an integer between 0 and 1000, a scaled difference score of two face embeddings. A smaller number indicates a greater difference between the two faces; a larger number indicates a greater similarity between the two faces.

The scoring mechanism is determined by the size of given embeddings (see `MetadataResponse` for more details).

C#

```
int GetMatchScore(IList<float> embedding1, IList<float> embedding2)
```

C++

```
paravision::Status GetMatchScore(
    int &score,
    const Embedding &embedding1,
    const Embedding &embedding2)
```

Go

```
GetMatchScore(embedding1 []float32, embedding2 []float32) (int,
error)
```

Java

```
int getMatchScore(
    List<Float> embedding1,
    List<Float> embedding2
) throws ClientException
```

Node

```
getMatchScore(embedding1: Array<!float>, embedding2: Array<!float>)
    => Promise<int>
```

Python

```
get_match_score(embedding1: Embedding, embedding2: Embedding) -> int
```

Ruby

```
get_match_score(embedding1, embedding2) -> int
```

Arguments

embedding1: The embedding for the first face. This can be generated either by calling process full image or process aligned face image with the `EMBEDDING` option.

embedding2: The embedding for the second face.

Return

score: An integer between 0 and 1000.

status: C++, `Status` object indicating call status. If no errors occured, `status.ok()` will be `true`.

Example

C#

```
// get the score between two embeddings
var score = client.GetMatchScore(face1.Embedding,
                                 face2.Embedding);
```

C++

```
// get the score between two embeddings
int score;
status = client.GetMatchScore(score, face1.embedding,
face2.embedding);
if(!status.ok())
{
    cerr << "Couldn't get match score! " << status.error_message
         << ": " << status.error_details << endl;
    exit(1);
}
```

Go

```
// get the score between two embeddings
score, err := client.GetMatchScore(face1.GetEmbedding(),
                                   face2.GetEmbedding())
if err != nil {
    log.Fatalf("Error getting match score: %v", err)
}
```

Java

```
// get the score between two embeddings
var score = client.getMatchScore(face1.getEmbeddingList(),
                                 face2.getEmbeddingList());
```

Node

```
// get the score between two embeddings
client.getMatchScore(face1.getEmbeddingList(),
face2.getEmbeddingList())
  .then(score => {
    return score;
  })
  .catch(err => {
    console.error(err);
  })
```

Python

```
# get the score between two embeddings
score = client.get_match_score(face1.embedding, face2.embedding)
```

Ruby

```
# get the score between two embeddings
score = client.get_match_score(face1.embedding, face2.embedding)
```

Get Similarity

Get Similarity takes a pair of embeddings and returns a float between 0 and 4, the the difference score of two face embeddings. A smaller number indicates a greater difference between the two faces; a larger number indicates a greater similarity between the two faces.

The scoring mechanism is determined by the size of given embeddings (see `MetadataResponse` for more details).

C#

```
float GetSimilarity(IList<float> embedding1, IList<float>
embedding2)
```

C++

```
paravision::Status GetSimilarity(
    float &similarity,
    const Embedding &embedding1,
    const Embedding &embedding2)
```

Go

```
GetSimilarity(embedding1 []float32, embedding2 []float32) (float32,
error)
```

Java

```
Float getSimilarity(
    List<Float> embedding1,
    List<Float> embedding2
) throws ClientException
```

Node

```
getSimilarity(embedding1: Array<!float>, embedding2: Array<!float>)
    => Promise<float>
```

Python

```
get_similarity(embedding1: Embedding, embedding2: Embedding) ->
float
```

Ruby

```
get_similarity(embedding1, embedding2) -> float
```

Arguments

embedding1: The embedding for the first face. This can be generated either by calling process full image or process aligned face image with the `EMBEDDING` option.

embedding2: The embedding for the second face.

Return

similarity: A float between 0 and 4.

status: C++, `Status` object indicating call status. If no errors occured, `status.ok()` will be `true`.

Example

C#

```
// get the similarity between two embeddings
var similarity = client.GetSimilarity(face1.Embedding,
                                      face2.Embedding);
```

C++

```
// get the similarity between two embeddings
float similarity;
status = client.GetSimilarity(similarity, face1.embedding,
face2.embedding);
if(!status.ok())
{
    cerr << "Couldn't get similarity! " << status.error_message
         << ": " << status.error_details << endl;
    exit(1);
}
```

Go

```
// get the similarity between two embeddings
similarity, err := client.GetSimilarity(face1.GetEmbedding(),
                                        face2.GetEmbedding())
if err != nil {
    log.Fatalf("Error computing similarity: %v", err)
}
```

Java

```
// get the similarity between two embeddings
var similarity = client.getSimilarity(face1.getEmbeddingList(),
                                      face2.getEmbeddingList());
```

Node

```
// get the similarity between two embeddings
client.getSimilarity(face1.getEmbeddingList(),
face2.getEmbeddingList())
  .then(similarity => {
    return similarity;
  })
  .catch(err => {
    console.error(err);
  })
```

Python

```
# get the similarity between two embeddings
similarity = client.get_similarity(face1.embedding, face2.embedding)
```

Ruby

```
# get the similarity between two embeddings
similarity = client.get_similarity(face1.embedding, face2.embedding)
```

Health Check

Query health of the Processor API.

Processor API implements gRPC Health checking protocol's ↗ Check method, it's available as a standard `/grpc.health.v1.Health/Check` RPC method.

To be used by infrastructure components like load balancers for purposes of monitoring the health of the container or for horizontal scaling.

Arguments

service: A string, ignored. Processor API exposes only one service, all checks return server's overall health status.

Return

status: An enum of type `ServingStatus`. It can have one of three possible values, `UNKNOWN`, `SERVING`, or `NOT_SERVING`.

Example

grpc_health_probe

`grpc_health_probe` ↗ is available inside the container for easy setup:

```
docker exec <PROCESSOR_CONTAINER_NAME> /bin/grpc_health_probe -
addr=localhost:50051
```

gRPCurl

gRPCurl ↗ can be used to query the health:

```
grpcurl -plaintext localhost:50051 grpc.health.v1.Health/Check
```

Process Identity Document (Experimental)

The document authentication introduced in Processor v7.9.0 is an experimental feature.

Experimental features may not offer the same level of accuracy or functionality as official features and are subject to change.

Only available with a compatible Regula docreader service ↗ (latest supported version is 7.5 ↗) . See PV_DOC_AUTH_URI for details.

Request parameters

Request is passed directly to Regula process endpoint, refer to their documentation for full request structure ↗.

Response

Response is a `ProcessIdentityDocumentResponse` whose `data` attribute is the response returned directly from Regula's process endpoint, refer to their documentation for full response structure ↗.

C#

```
ProcessIdentityDocumentResponse ProcessIdentityDocument(
    string requestBody,
    float? timeout = null)
```

Async analogue is also available since v6.0.1:

```
async Task<pb.ProcessIdentityDocumentResponse>
ProcessIdentityDocumentAsync(
    string requestBody,
    float? timeout = null)
```

C++

```
paravision::Status ProcessIdentityDocument(
    ProcessIdentityDocumentResponse &out_response,
    const string &request_body,
    float timeout = -1)
```

Go

```
ProcessIdentityDocument(
    ctx context.Context,
    request_body string)
(*pb.ProcessIdentityDocumentResponse, error)
```

Java

```
ProcessIdentityDocumentResponse processIdentityDocument(
            String requestBody,
            float timeout)
throws ClientException
```

Node

```
processIdentityDocument(request_body, timeout = null)
    => Promise<ProcessIdentityDocumentResponse>
```

Python

```
process_identity_document(
    self,
    request_body: str,
    timeout: Optional[float] = None
) -> ProcessIdentityDocumentResponse:
```

Ruby

```
process_identity_document(request_body, timeout: nil)
```

Arguments

ctx *(Go only)*: `Context` ↗ to be used with underlying gRPC calls.

request_body: A string representing the json body of the http request that gets passed directly to Regula process endpoint, refer to their documentation for full request structure ↗.

timeout *(not available in Go)*: A float value, specifying the timeout in seconds for the underlying gRPC calls. No timeout is specified by a null value (where the language allows), or by -1 in the languages where float is not nullable.

Return

response: A `ProcessIdentityDocumentResponse` object containing the response from the Regula API. Refer to their documentation for the full response structure ↗. In some cases where Regula provides a client (C#, Java, Javascript, Python ↗), this response can be parsed much more easily by parsing the response with their `RecognitionResponse` object, examples for appropriate languages will be shown below.

status: C++, `Status` object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
using Newtonsoft.Json;
using Newtonsoft.Json.Linq;
using Regula.DocumentReader.WebClient.Model.Ext;
using Regula.DocumentReader.WebClient.Model;

// Build the request body following the structure of regula process
endpoint
// https://dev.regulaforensics.com/DocumentReader-web-
openapi/#tag/process
byte[] idImageData = File.ReadAllBytes("path/to/id.jpg");
string base64Image = Convert.ToBase64String(idImageData);

JObject requestBody = new JObject(
    new JProperty("List", new JArray(
        new JObject(
            new JProperty("ImageData", new JObject(
                new JProperty("image", base64Image)
            ))
        )
    )),
    new JProperty("processParam", new JObject(
        new JProperty("scenario", "FullAuth")
    ))
);

// Run doc auth
var result = client.ProcessIdentityDocument(requestBody.ToString(),
timeout);

// Process response using Regula C# client
// https://github.com/regulaforensics/DocumentReader-web-csharp-
client
ProcessResponse processResponse =
JsonConvert.DeserializeObject<ProcessResponse>(result.Data);
RecognitionResponse docAuthResponse = new
RecognitionResponse(processResponse);

if (docAuthResponse.Status().OverallStatus != 1) {
    throw new Exception("Status is not OK");
}

var docNumber =
docAuthResponse?.Text()?.GetField(TextFieldType.DOCUMENT_NUMBER)?.Ge
tValue(Source.VISUAL) ?? "";
if (docNumber != "RB1234567") {
    throw new Exception("Unexpected Doc Number");
}
```

C++

```
// Load the image in the correct format
string id_image_data;
{
    auto ss = ostringstream{};
    ifstream file("path/to/id.jpg", ifstream::in |
ifstream::binary);
    if(!file)
    {
        cerr << "Couldn't open file please make sure it exists." <<
endl;
        return 6;
    }

    ss << file.rdbuf();

    id_image_data = encode64(ss.str());
}

// Build the request body following the structure of regula process
endpoint
// https://dev.regulaforensics.com/DocumentReader-web-
openapi/#tag/process
ProcessIdentityDocumentResponse resp;
nlohmann::json request_body_json;
request_body_json["processParam"] = {{"scenario", "FullAuth"}};
request_body_json["List"] = {{{"ImageData", {{"image",
id_image_data}}}}};

// Process identity document
status = cli.ProcessIdentityDocument(resp,
request_body_json.dump());
if(!status.ok())
{
    cerr << "Processing identity document failed! " <<
status.error_message << endl;
    return 3;
}

// Parse the raw json response
// Response body structure defined in regula documentation
// https://dev.regulaforensics.com/DocumentReader-web-
openapi/#tag/process
nlohmann::json doc_auth_result = nlohmann::json::parse(resp.data());
if (doc_auth_result.contains("ContainerList")) {
    const auto& containers = doc_auth_result["ContainerList"]
["List"];
    for (const auto& container : containers) {
        int result_type = container["result_type"];
```

```
        int result_type = container["result_type"];
        switch (result_type) {
            case IDVResponse::ResultType::TEXT: {
                for (const auto& text_field : container["Text"]
["fieldList"]) {
                    int field_type = text_field["fieldType"];
                    auto field_value = text_field["value"];

                    switch (field_type) {
                        case IDVResponse::TextFieldType::LAST_NAME:
{
                            if (field_value != "CITIZEN") {
                                cerr << "Doc Auth check failed!
Unexpected Last Name" << endl;
                                return RET_ERR;
                            }
                            break;
                        }

                    }
                }
                break;
            }
            case IDVResponse::ResultType::STATUS: {
                auto status = container["Status"]["overallStatus"];
                if (status != 1) {
                    cerr << "Status not OK" << endl;
                    return RET_ERR;
                }
                break;
            }
        }
    }
}
```

Go

```
// Build request body according to regula api /process endpoint
// https://dev.regulaforensics.com/DocumentReader-web-
openapi/#tag/process/operation/api-process
idImageData, err := ioutil.ReadFile(*idImagePath)
if err != nil {
    log.Fatalf("Couldn't read input id-image: %v", err)
}

encodedImage := base64.StdEncoding.EncodeToString(idImageData)

imageDataPayload := map[string]interface{}{
    "ImageData": map[string]interface{}{
        "image": encodedImage,
    },
}

body := map[string]interface{}{
    "List": []interface{}{imageDataPayload},
    "processParam": map[string]interface{}{
        "scenario": "FullAuth",
    },
}

requestBody, _ := json.Marshal(body)

// Run doc auth
docAuthResponse, err := client.ProcessIdentityDocument(ctx,
string(requestBody))
if err != nil {
    log.Fatalf("Error processing identity document: %v", err)
}

// Parse the raw json respons according to regula /process endpoint
output
// https://dev.regulaforensics.com/DocumentReader-web-
openapi/#tag/process/operation/api-process
var docAuthResponseData map[string]interface{}
json.Unmarshal([]byte(docAuthResponse.Data), &docAuthResponseData)

containerList, ok := docAuthResponseData["ContainerList"].
(map[string]interface{})["List"].([]interface{})
if !ok {
    log.Fatal("Error: ContainerList or List not found or not the
correct type")
}

for _, containerObj := range containerList {
    container := containerObj.(map[string]interface{})
```

```
    statusObj, statusExists := container["Status"].
(map[string]interface{})
    if !statusExists {
        // if "Status" doesn't exist, skip to the next container
        continue
    }

    detailsOpticalObj := statusObj["detailsOptical"].
(map[string]interface{})
    overallStatus := detailsOpticalObj["overallStatus"].(float64)
    if overallStatus != 1 {
        log.Fatalf("Status not OK")
    }
}
```

Java

```
// Build Doc Auth Request according to regula /process request
// https://dev.regulaforensics.com/DocumentReader-web-
openapi/#tag/process/operation/api-process
byte[] idImageData =
Files.readAllBytes(Paths.get("path/to/id.jpg"));
String base64Image =
Base64.getEncoder().encodeToString(idImageData);
JSONObject jsonImageData = new JSONObject();
jsonImageData.put("image", base64Image);

JSONObject imageDataWrapper = new JSONObject();
imageDataWrapper.put("ImageData", jsonImageData);

JSONArray listArray = new JSONArray();
listArray.put(imageDataWrapper);

JSONObject processParam = new JSONObject();
processParam.put("scenario", "FullAuth");

JSONObject body = new JSONObject();
body.put("List", listArray);
body.put("processParam", processParam);

ProcessIdentityDocumentResponse idResponse =
client.processIdentityDocument(body.toString(), timeout);

// Parse Doc Auth Response according to regula /process response
// https://dev.regulaforensics.com/DocumentReader-web-
openapi/#tag/process/operation/api-process
JSONObject responseObject = new JSONObject(idResponse.getData());
JSONArray containerList =
responseObject.getJSONObject("ContainerList").getJSONArray("List");

boolean statusPresent = false;
for (int i = 0; i < containerList.length(); i++) {
    JSONObject container = containerList.getJSONObject(i);
    if (container.has("Status")) {
        statusPresent = true;
        JSONObject status = container.getJSONObject("Status");
        int overallStatus =
status.getJSONObject("detailsOptical").getInt("overallStatus");
        if (overallStatus != 1) {
            throw new Exception("Status not OK");
        }
    }
}
```

Node

```
const { TextFieldType, Response } =
require('@regulaforensics/document-reader-webclient');
const { DOCUMENT_NUMBER } = TextFieldType;

// Read data from image file
imageData = fs.readFileSync("path/to/id.jpg");

// Build the request according to regula /process request body
// https://dev.regulaforensics.com/DocumentReader-web-
openapi/#tag/process/operation/api-process
const body = {
  List: [
    {
      ImageData: {
        image: Buffer.from(imageData).toString("base64"),
      },
    },
  ],
  processParam: { scenario: "FullAuth" },
};

let docAuthResponse = await
client.processIdentityDocument(JSON.stringify(body), timeout);
const parsedResponse = new Response(JSON.parse(docAuthResponse));

// the overall status should fail for the test document
if (parsedResponse.status?.overallStatus != 1) {
  throw new Error("Status not OK");
}
```

Python

```
import base64
import json

from regula.documentreader.webclient import RecognitionResponse,
TextFieldType
from regula.documentreader.webclient.gen.api_client import ApiClient

with open("path/to/id.jpg", "rb") as f:
    image = f.read()

# Build the json request body accordint to regula openapi request
structure
# https://dev.regulaforensics.com/DocumentReader-web-
openapi/#tag/process/operation/api-process
body = {"List": [{"ImageData": {"image":
base64.b64encode(image).decode("utf-8")}}], "processParam":
{"scenario": "FullAuth"}}

result =
client.process_identity_document(request_body=json.dumps(body))

# use regula to deserialize the json response
response_object = ApiClient().deserialize(result, "ProcessResponse")
response = RecognitionResponse(response_object)

# Pull sample text field from the document
print(response.text.get_field(TextFieldType.DOCUMENT_NUMBER).get_val
ue())
```

Ruby

```
id_image_data = File.binread("path/to/id.jpg")
encoded_id_image = Base64.strict_encode64(id_image_data)

# build request body according to regula /process request
# https://dev.regulaforensics.com/DocumentReader-web-
openapi/#tag/process/operation/api-process
request_body = {
  "List" => [
    {
      "ImageData" => {
        "image" => encoded_id_image
      }
    }
  ],
  "processParam" => {
    "scenario" => "FullAuth"
  }
}

json_request_body = JSON.generate(request_body)

# send doc auth request
response = client.process_identity_document(
  json_request_body,
  timeout: TIMEOUT,
)

# Parse raw json response according to regula /process response
# https://dev.regulaforensics.com/DocumentReader-web-
openapi/#tag/process/operation/api-process
parsed_id_result = JSON.parse(response.data)
container_list = parsed_id_result.dig("ContainerList", "List")
if container_list
  container_list.each do |container|
    if container.key?("Status")
      status = container["Status"]
      overall_status = status["detailsOptical"]["overallStatus"]

      if overall_status != 1
        raise "Status not OK"
      end
    end
  end
end
```

Get Usage Report (Experimental)

> The usage reporting introduced in Processor v7.9.0 is an experimental feature.
>
> *Experimental features may not offer the same level of accuracy or functionality as official features and are subject to change.*

The GetUsageReport endpoint returns a report log of the features transacted by the Processor API.

C#

```
pb.UsageReportResponse GetUsageReport(float? timeout)
```

Async analogue is also available since v6.0.1:

```
async Task<pb.UsageReportResponse> GetUsageReportAsync(float?
timeout)
```

C++

```
paravision::Status GetUsageReport(UsageReportResponse &usage_report,
float timeout = -1)
```

Go

```
GetUsageReport(ctx context.Context) (*pb.UsageReportReponse, error)
```

Java

```
UsageReportResponse getUsageReport(float timeout = -1) throws
ClientException
```

Node

```
getUsageReport(timeout: number = null) =>
Promise<UsageReportResponse>
```

Python

```
get_usage_report(timeout: Optional[float] = None) ->
UsageReportResponse
```

Ruby

```
get_usage_report(timeout: nil) -> UsageReportResponse
```

Arguments

ctx *(Go only)*: `Context` ↗ to be used with underlying gRPC calls.

timeout *(not available in Go)*: A float value, specifying the timeout in seconds for the underlying gRPC calls. No timeout is specified by a null value (where the language allows), or by -1 in the languages where float is not nullable.

Return

metadata: A `UsageReportResponse` object, containing an encoded report string field with logs of the features transacted by the Processor API.

status: C++, `Status` object indicating call status. If no errors occured, `status.ok()` will be `true`.

Example

C#

```
var usage_report = client.GetUsageReport();
```

C++

```
UsageReportResponse usage_report;
auto status = client.GetUsageReport(usage_report);
if(!status.ok())
{
    cerr << "Error getting metadata!" << status.error_message
         << ": " << status.error_details << endl;
    exit(1);
}
```

Go

```
ctx, cancel := context.WithCancel(context.Background())
defer cancel()

usage_report, err := client.GetUsageReport(ctx)
if err != nil {
    log.Fatalf("Error getting usage report, %v", err)
}
```

Java

```
var usage_report = client.getUsageReport();
```

Node

```
client.getUsageReport()
  .then(usageReport => {
    console.log(usageReport);
  })
  .catch(err => {
    console.error(err);
  })
```

Python

```
usage_report = client.get_usage_report()
```

Ruby

```
usage_report = client.get_usage_report
```

HTTP Usage

Setup

The Processor API container is capable of exposing an HTTP REST interface for all its features. The HTTP server is not enabled by default, you must set the `PV_HTTP_INTERFACE` environment variable of the Processor API container to `on` to activate it. You can do this e.g. by adding `-e PV_HTTP_INTERFACE=on` to the `docker run` command when starting the container. Additionally, you must expose the HTTP port to outside of the container, which can be done with `-p 8081:8081`. Thus, to use the REST interface, you need to start the Processor container like this:

```
docker run -dt --rm --name processor-api -e PV_HTTP_INTERFACE=on -p
50051:50051 \
    -p 8081:8081 containers.paravision.ai/processor/processor:<tag>
```

Refer to installation for details on starting processor container.

CORS

Cross-Origin Resource Sharing (CORS) can be enabled for the HTTP server by setting the `PV_CORS_ORIGINS` environment variable. CORS is disabled by default. To enable it, set the `PV_CORS_ORIGINS` environment variable to a comma separated list of origins that should be allowed. For origin values, `*` and `null` can also be used. For example, `-e PV_CORS_ORIGINS=*` can be used with the `docker run` command when starting the container to enable CORS for all origins.

HTTP basic authentication

HTTP basic authentication ↗ can be enabled for the HTTP server by setting the `PV_HTTP_AUTH*` environment variable. It is disabled by default. To enable it, set the `PV_HTTP_AUTH=on` and provide a username and a password in `PV_HTTP_AUTH_USER`

and `PV_HTTP_AUTH_PASSWORD` environment variables. There is also `PV_HTTP_AUTH_PASSWORD_FILE` environment variable available if you wish to use a file within the container instead of an environment variable to provide the password to the HTTP server (like, for example, a Docker secret).

You can test the authentication with `curl` by providing a `-u USER:PASSWORD` flag to the command. For example:

```
curl -u USER:PASSWORD -s -X GET http://localhost:8081/v7/health_check |
jq .
```

If credentials are invalid or missing, the server will respond with status code `401 Unauthorized` and the following message:

```
{
  "message": "Unauthenticated",
  "code": 16,
  "details": []
}
```

Common features

Unless otherwise stated, all arguments for POST requests need to be passed as JSON objects (`application/json`). URL-encoded form fields (`application/x-www-form-urlencoded`) are not supported.

Data is returned in response body as JSON-formatted objects (`application/json`).

All endpoints indicate errors with a relevant HTTP response code, and additionally a message describing error details is available as a JSON-formatted object in response body, with following fields:

message: A human-readable description of the error.

code: An internal error type number.

details: Reserved for future use.

An example error response looks like this:

```
{
  "message": "parsing field \"image\": illegal base64 data at input
byte 462",
  "code": 3,
  "details": []
}
```

Process Full Image

```
POST /v7/process_full_image
```

Process full image takes an uncropped image and returns information about the faces in the image.

Request parameters

image: Base64-encoded bytes of the image to process.

outputs: A list of `ProcessFullImageOptions`, the attributes to be included in the returned `Face` objects.

find_most_prominent_face: A boolean value, describes whether `most_prominent_face_idx` should be returned (default False).

liveness_validness_parameters: a `ProcessFullImageLivenessValidnessParameters` object, used to specify validness parameters for liveness.

ages_v2_validness_parameters: a `ProcessFullImageAgeValidnessParameters` object, used to specify validness parameters for liveness.

deepfake_validness_parameters: a `ProcessFullImageDeepfakeValidnessParameters` object, used to specify validness parameters for deepfake.

image_source: a string to specify the source of the image. Allowed values are `mobile`, `webcam` and `unknown`. Default is `unknown`.

> From v7.7.0 onward, Face objects from detection functions are now sorted based on prominence (largest face by area). Therefore, the most prominent face is always at index 0.

> In order to use the `LIVENESS` and `LIVENESS_VALIDNESS` outputs you must be running the liveness enabled processor container. If you request liveness from a container that does not have liveness enabled the request will fail.

> In order to use the `AGES_V2` and `AGES_V2_VALIDNESS` outputs you must be running the ages_v2 enabled processor container. If you request ages_v2 from a container that does not have ages_v2 enabled the request will fail.

> In order to use the `DEEPFAKE` and `DEEPFAKE_VALIDNESS` outputs you must be running the deepfake enabled processor container. If you request deepfake from a container that does not have deepfake enabled the request will fail.

Response

A JSON object with fields:

faces: A list of `Faces` found in the image, each with the requested attributes attached.

> **Note**: While each returned face contains fields for attributes that were not requested, they will be empty or invalid.

most_prominent_face_idx: Return index of most prominent face if requested using the `find_most_prominent_face` parameter (if not -1 is returned).

Example-1

> **Note**: due to limited command-line size, this example only works with small images. To process large images, use a HTTP client library, or build your request otherwise.

```
echo "{
    \"image\": \"$(base64 image.jpg)\",
    \"outputs\": [
      \"BOUNDING_BOX\",
      \"ALIGNED_FACE_IMAGE\",
      \"QUALITY\",
      \"MASK\"
    ],
    \"find_most_prominent_face\": false
  }" | curl -s -X POST http://localhost:8081/v7/process_full_image -d@-
| jq .
```

```
{
  "faces": [
    {
      "bounding_box": {
        "origin": {
          "x": 15.123,
          "y": 15.4231
        },
        "width": 64.55429,
        "height": 79.70598
      },
      "landmarks": null,
      "embedding": [],
      "ages": null,
      "genders": null,
      "aligned_face_image": "iVBORw0KGgoAAAA [...]",
      "acceptability": 0.751821,
      "quality": 0.629021,
      "mask": 0.002408,
      "liveness": null,
      "liveness_validness": null
    },
    ... more faces
  ],
  "most_prominent_face_idx": -1
}
```


Example-2

```
echo "{
    \"image\": \"$(base64 image.jpg)\",
    \"outputs\": [
      \"EMBEDDING\",
      \"GENDERS\",
      \"AGES\",
      \"LANDMARKS\"
    ],
    \"find_most_prominent_face\": true
  }" | curl -s -X POST http://localhost:8081/v7/process_full_image -d@-
| jq .
```

```
{
  "faces": [
    {
      "bounding_box": null,
      "landmarks": {
        "left_eye": {
          "x": 27.31415,
          "y": 34.9265
        },
        "right_eye": {
          "x": 43.3589,
          "y": 35.7932
        },
        "nose": {
          "x": 35.3846,
          "y": 44.2643
        },
        "left_mouth": {
          "x": 28.3832,
          "y": 53.7950
        },
        "right_mouth": {
          "x": 42.2884,
          "y": 53.1971
        }
      },
      "embedding": [
        0.036385342,
        -0.008870514,
        0.13322286,
        0.0139650935,
        0.06830906,
        0.15191919,
        -0.080664076,
        [...]
      ],
      "ages": {
        "confidence": 0.516929,
        "value": "31-40",
        "distribution": [
          {
            "confidence": 0.000253,
            "value": "2-12"
          },
          {
            "confidence": 0.001715,
            "value": "13-19"
          },
```

```
          {
            "confidence": 0.043234,
            "value": "20-30"
          },
          [...]
        ]
      },
      "genders": {
        "confidence": 0.999565,
        "value": "male",
        "distribution": [
          {
            "confidence": 0.999565,
            "value": "male"
          },
          {
            "confidence": 0.000435,
            "value": "female"
          }
        ]
      },
      "aligned_face_image": "",
      "acceptability": 0,
      "quality": 0,
      "mask": 0,
      "liveness": null,
      "liveness_validness": null
    },
    ... more faces
  ],
  "most_prominent_face_idx": 2
}
```


Example-3

```
echo "{
    \"image\": \"$(base64 image.jpg)\",
    \"outputs\": [
      \"LIVENESS\",
      \"LIVENESS_VALIDNESS\"
    ],
    \"find_most_prominent_face\": false
  }" | curl -s -X POST http://localhost:8081/v7/process_full_image -d@-
| jq .
```

```
{
  "faces": [
    {
      "bounding_box": null,
      "landmarks": null,
      "embedding": [],
      "ages": null,
      "genders": null,
      "aligned_face_image": "",
      "acceptability": 0,
      "quality": 0,
      "mask": 0,
      "liveness": {
        "liveness_probability": 0.98
      },
      "liveness_validness": {
        "sharpness": 0.75,
        "quality": 0.95,
        "acceptability": 0.95,
        "frontality": 0.99,
        "image_average_pixel_value": 32.1,
        "image_bright_pixel_pct": 23.1,
        "image_dark_pixel_pct": 55.0,
        "mask_probability": 0.01,
        "height_pct": 88.0,
        "position_pcts": [101.1, 102.2, 103.3, 104.4],
        "is_valid": true,
        "feedback": []
      }
    }
  ],
  "most_prominent_face_idx": -1
}
```


Example-4

```
echo "{
    \"image\": \"$(base64 image.jpg)\",
    \"outputs\": [
      \"BOUNDING_BOX\",
      \"DEEPFAKE\"
    ],
    \"find_most_prominent_face\": false
  }" | curl -s -X POST http://localhost:8081/v7/process_full_image -d@-
| jq .
```


```
{
  "faces": [
    {
      "acceptability": 0.9996218085289001,
      "aligned_face_image": "",
      "bounding_box": {
        "height": 1472.57470703125,
        "origin": {
          "x": 668.65771484375,
          "y": 819.1174926757812
        },
        "width": 1002.2116088867188
      },
      "deepfake": {
        "detailed_scores": {
          "face_swap_x": -1,
          "face_swap_y": -1,
          "synthetic": -1
        },
        "realness": 0.9783934354782104,
        "type": "SYNTHETIC"
      },
      "deepfake_realness": 0.9783934354782104,
      "embedding": [],
      "embedding_template": "",
      "landmarks": null,
      "quality": 0
    }
  ],
  "most_prominent_face_idx": -1,
  "success": true
}
```

Example-5

```
echo "{
    \"image\": \"$(base64 image.jpg)\",
    \"outputs\": [
      \"BOUNDING_BOX\",
      \"AGES_V2\",
      \"AGES_V2_VALIDNESS\"
    ],
    \"find_most_prominent_face\": false
  }" | curl -s -X POST http://localhost:8081/v7/process_full_image -d@-
| jq .
```

```
{
  "faces": [
    {
      "acceptability": 0.9996218085289001,
      "ages_v2": 17.521848678588867,
      "ages_v2_validness": {
        "acceptability": 0.9996218085289001,
        "feedback": [],
        "frontality": 94,
        "image_average_pixel_value": 131.04904174804688,
        "is_valid": true,
        "mask_probability": 0.007835040800273418,
        "position_pcts": [
          0.2887122631072998,
          0.2652582824230194,
          0.72144615650177,
          0.7421281933784485
        ],
        "quality": 0.8223484754562378,
        "sharpness": 0.8345630168914795
      },
      "aligned_face_image": "",
      "attributes_images": "",
      "bounding_box": {
        "height": 1472.574462890625,
        "origin": {
          "x": 668.6575927734375,
          "y": 819.1175537109375
        },
        "width": 1002.211669921875
      },
      "deepfake": {
        "realness": -1,
        "type": "UNKNOWN"
      },
      "deepfake_realness": -1,
      "embedding": [],
      "embedding_template": "",
      "landmarks": null,
      "quality": 0
    }
  ],
  "most_prominent_face_idx": -1,
  "success": true
}
```

Example-6

```
echo "{
    \"image\": \"$(base64 image.jpg)\",
    \"outputs\": [
      \"BOUNDING_BOX\",
      \"DEEPFAKE\",
      \"DEEPFAKE_VALIDNESS\"
    ],
    \"find_most_prominent_face\": false
  }" | curl -s -X POST http://localhost:8081/v7/process_full_image -d@-
| jq .
```

```
{
  "faces": [
    {
      "acceptability": 0.9996218085289001,
      "ages_v2": -1,
      "aligned_face_image": "",
      "attributes_images": "",
      "bounding_box": {
        "height": 1472.57470703125,
        "origin": {
          "x": 668.65771484375,
          "y": 819.1174926757812
        },
        "width": 1002.21142578125
      },
      "deepfake": {
        "detailed_scores": {
          "face_swap_x": -1,
          "face_swap_y": -1,
          "synthetic": -1
        },
        "realness": 0.9745436310768127,
        "type": "FACE_SWAP"
      },
      "deepfake_realness": 0.9745436310768127,
      "deepfake_validness": {
        "acceptability": 0.9996218085289001,
        "feedback": [],
        "is_valid": true,
        "position_pcts": [
          0.2887122631072998,
          0.2652582824230194,
          0.72144615650177,
          0.7421281933784485
        ],
        "quality": 0.8223484754562378
      },
      "embedding": [],
      "embedding_template": "",
      "landmarks": null,
      "quality": 0
    }
  ],
  "most_prominent_face_idx": -1,
  "success": true
}
```

Process Prepared Face Image

```
POST /v7/process_prepared_face_image
```

Process aligned face image takes a single cropped and aligned face image and returns information about the face.

Request parameters

image: Base64-encoded bytes of the aligned image to process for EMBEDDING option. An aligned image can be generated using process full image with the `ALIGNED_FACE_IMAGE` option, or by accessing the `recognition_input_image` image of a Face emitted by our Streaming container ↗.

attributes_images: An array of images to process for all attributes options. This can be generated by calling process full image with the `ATTRIBUTES_IMAGES` option, or by accessing the `attributes_images` images of a Face emitted by our Streaming container ↗.

deepfake_images: An array of images to process for all deepfake options. This can be generated by calling process full image with the `DEEPFAKE_IMAGES` option.

liveness_image: Base64-encoded bytes of the liveness image to process for all liveness options. This can be generated by calling process full image with the `LIVENESS_IMAGE` option.

ages_v2_image: Base64-encoded bytes of the ages_v2 image to process for ages_v2 options. This can be generated by calling process full image with the `AGES_V2_IMAGE` option.

outputs: A list of `ProcessPreparedFaceImageOptions`, the attributes to be included in the returned `Face` object.

The client application should employ mechanisms to ensure the correct image(s) are used for inputs: `image`, `attributes_images`, `deepfake_images`, `liveness_image`, `ages_v2_image`. The incorrect image can sometimes lead to erroneous results.

Response

A JSON object with a single field:

face: A `Face` with the requested attributes attached.

Note: While each returned face contains fields for attributes that were not requested, they will be empty or invalid.

Example

Note: due to limited command-line size, this example only works with small images. To process large images, use a HTTP client library, or build your request otherwise.

```
echo "{
    \"image\": \"$(base64 image.jpg)\",
    \"attributes_images\": [\"$(base64 image.jpg)\", \"$(base64
image.jpg)\"],
    \"outputs\": [
      \"EMBEDDING\",
      \"AGES\",
      \"GENDERS\"
    ]
  }" | curl -s -X POST
http://localhost:8081/v7/process_prepared_face_image -d@- | jq .
```

```
{
  "face": {
    "bounding_box": null,
    "landmarks": null,
    "embedding": [
      0.036385342,
      -0.008870514,
      0.13322286,
      0.0139650935,
      0.06830906,
      0.15191919,
      -0.080664076,
      [...]
    ],
    "ages": {
      "confidence": 0.516929,
      "value": "31-40",
      "distribution": [
        {
          "confidence": 0.000253,
          "value": "2-12"
        },
        {
          "confidence": 0.001715,
          "value": "13-19"
        },
        {
          "confidence": 0.043234,
          "value": "20-30"
        },
        [...]
      ]
    },
    "genders": {
      "confidence": 0.999565,
      "value": "male",
      "distribution": [
        {
          "confidence": 0.999565,
          "value": "male"
        },
        {
          "confidence": 0.000435,
          "value": "female"
        }
      ]
    },
    "aligned_face_image": "",
```

```
    "acceptability": 0,
    "quality": 0,
    "mask": 0
  }
}
```


Compare Most Prominent Faces

```
POST /v7/compare_most_prominent_faces
```

Compare most prominent faces takes two images, finds the most prominent face on each of them, and then returns a `match_score` between these faces and some basic information about them.

Request parameters

image1: Base64-encoded bytes of the first image to process.

image2: Base64-encoded bytes of the second image to process.

Response

A JSON object with fields:

match_score: Match score of the two most prominent faces found on the input images. An integer between 0 and 1000. The greater the number, the more similar the most prominent faces are.

image1_face: The most prominent `Face` found in the first image, with some basic information attached.

image2_face: The most prominent `Face` found in the second image, with some basic information attached.

> **Note**: Only `acceptability`, `quality`, `landmarks` and `bounding_box` will be calculated for the most prominent faces.

Example

```
echo "{
    \"image1\": \"$(base64 image1.jpg)\",
    \"image2\": \"$(base64 image2.jpg)\"
  }" | curl -s -X POST
http://localhost:8081/v7/compare_most_prominent_faces -d@- | jq .
```

```
{
  "match_score": 951,
  "image1_face": {
    "bounding_box": {
      "origin": {
        "x": 197.64912,
        "y": 108.458275
      },
      "width": 158.59282,
      "height": 220.37053
    },
    "landmarks": {
      "left_eye": {
        "x": 230.89919,
        "y": 187.34639
      },
      "right_eye": {
        "x": 304.65848,
        "y": 183.03218
      },
      "nose": {
        "x": 263.04437,
        "y": 236.64716
      },
      "left_mouth": {
        "x": 242.20769,
        "y": 270.68204
      },
      "right_mouth": {
        "x": 303.14255,
        "y": 266.90717
      }
    },
    "embedding": [],
    "ages": null,
    "genders": null,
    "aligned_face_image": "",
    "acceptability": 0.99914384,
    "quality": 0.89508545,
    "mask": 0
  },
  "image2_face": {
    "bounding_box": {
      "origin": {
        "x": 197.64912,
        "y": 108.458275
      },
      "width": 158.59282,
```

```
        "height": 220.37053
      },
      "landmarks": {
        "left_eye": {
          "x": 230.89919,
          "y": 187.34639
        },
        "right_eye": {
          "x": 304.65848,
          "y": 183.03218
        },
        "nose": {
          "x": 263.04437,
          "y": 236.64716
        },
        "left_mouth": {
          "x": 242.20769,
          "y": 270.68204
        },
        "right_mouth": {
          "x": 303.14255,
          "y": 266.90717
        }
      },
      "embedding": [],
      "ages": null,
      "genders": null,
      "aligned_face_image": "",
      "acceptability": 0.99914384,
      "quality": 0.89508545,
      "mask": 0
    }
}
```


Get Metadata

```
GET /v7/get_metadata
```

Get metadata returns the name of the recognition model and the name of the engine that your processor container is running in addition to the weight and bias being used.

Request parameters

None.

Response

A `MetadataResponse` object, containing the name of the model and the name of the engine your processor container is running in addition to the weight and bias.

Example

```
curl -s -X GET http://localhost:8081/v7/get_metadata | jq .
```

```
{
  "active_embedding_size": 257,
  "ages_v2_default_thresholds": {
    "fail_fast": true,
    "image_boundary_height_pct": 1,
    "image_boundary_width_pct": 1,
    "image_illumination_control": 50,
    "max_face_mask_probability": 0.5,
    "max_face_size_pct": 1,
    "min_face_acceptability": 0.15000000596046448,
    "min_face_frontality": 60,
    "min_face_quality": 0.5,
    "min_face_sharpness": 0.15000000596046448,
    "min_face_size": 100
  },
  "ages_v2_model_name": "age",
  "ages_v2_model_version": "2.0.0",
  "ages_v2_sdk_version": "1.2.0",
  "app_version": "7.8.0",
  "attributes_model_name": "attributes",
  "attributes_model_version": "2.0.1",
  "attributes_sdk_version": "2.1.2",
  "deepfake_default_thresholds": {
    "fail_fast": true,
    "min_face_acceptability": 0.15000000596046448,
    "min_face_quality": 0.5,
    "min_face_size": 50
  },
  "deepfake_model_name": "deepfake",
  "deepfake_model_version": "2.0.0",
  "deepfake_sdk_version": "2.3.0",
  "engine_name": "openvino",
  "enhanced_scoring_parameters": {
    "bias": -0.5,
    "sizes": [
      257,
      513,
      1025
    ],
    "weight": 2.3499999046325684
  },
  "liveness2d_model_name": "liveness2d",
  "liveness2d_model_version": "1.0.0",
  "liveness2d_sdk_version": "0.10.0",
  "liveness_default_thresholds": {
    "fail_fast": true,
    "image_boundary_height_pct": 0.800000011920929,
    "image_boundary_width_pct": 0.800000011920929,
```

```
    "image_illumination_control": 50,
    "max_face_mask_probability": 0.5,
    "max_face_roll_angle": 45,
    "max_face_size_pct": 0.7200000286102295,
    "min_face_acceptability": 0.15000000596046448,
    "min_face_frontality": 70,
    "min_face_quality": 0.5,
    "min_face_sharpness": 0.15000000596046448,
    "mobile_min_face_size": 175,
    "webcam_min_face_size": 100
  },
  "recognition_model_name": "fast",
  "recognition_model_version": "1.1.0",
  "recognition_sdk_version": "9.2.0",
  "standard_scoring_parameters": {
    "bias": -5.349999904632568,
    "sizes": [
      256,
      512,
      1024
    ],
    "weight": 2.200000047683716
  }
}
```


Health Check

```
GET /v7/health_check
```

Query health of the Processor API.

This endpoint exposes gRPC Health checking protocol's ↗ Check method.

To be used by infrastructure components like load balancers for purposes of monitoring the health of the container or for horizontal scaling.

Arguments

service: A string, ignored. Processor API exposes only one service, all checks return server's overall health status.

Response

A single JSON object with the following field:

status: An enum of type `ServingStatus`. It can have one of three possible values, `UNKNOWN`, `SERVING`, or `NOT_SERVING`.

Example

```
curl -s -X GET http://localhost:8081/v7/health_check | jq .
```

```
{
  "status": "SERVING"
}
```

Get Scores

```
POST /v7/get_scores
```

This function returns the match score and a similarity score for two embeddings.

Request Parameters

embedding1: A base64 encoded bytes of the first embedding (embedding_template from process_full_image)

embedding2: A base64 encoded bytes of the first embedding (embedding_template from process_full_image)

get_similarity: Whether or not to return the similarity score

get_match: Whether or not to return the match score

Example

```
echo "{
    \"embedding1\": \"$(base64 face1.bin)\",
    \"embedding2\": \"$(base64 face2.bin)\",
    \"get_similarity\": true,
    \"get_match\": true
  }" | curl -s -X POST http://localhost:8081/v7/get_scores -d@- | jq .
```

```
{
    "similarity": 123.456,
    "match": 600,
}
```

Get Match Score

See Get Scores

Get Similarity

See Get Scores

Process Identity Document (Experimental)

> The document authentication introduced in Processor v7.9.0 is an experimental feature.

Experimental features may not offer the same level of accuracy or functionality as official features and are subject to change.

```
POST /v7/process_identity_document
```

Only available with Regula ↗ license and setup. See PV_DOC_AUTH_URI for details.

Request parameters

Request is passed directly to Regula process endpoint, refer to their documentation for full request structure ↗.

Response

Response is returned directly from Regula process endpoint, refer to their documentation for full response structure ↗.

Example

```
echo "{
    \"processParam\": {\"scenario\": \"FullAuth\"},
    \"List\": [{\"ImageData\": {\"image\": \"$(base64
path/to/id.jpg)\"}}]
}" | curl -s -X POST http://localhost:8081/v7/process_identity_document
-d@- -H "Content-Type: application/json" -H "Expect:" | jq .
```

Get UsageReport (Experimental)

The usage reporting introduced in Processor v7.9.0 is an experimental feature.

> *Experimental features may not offer the same level of accuracy or functionality as official features and are subject to change.*

```
GET /v7/get_usage_report
```

The GetUsageReport endpoint returns a report log of the features transacted by the Processor API.

Request parameters

None.

Response

A `UsageReportResponse` object, containing an encoded report string field with logs of the features transacted by the Processor API.

Example

```
curl -s -X GET http://localhost:8081/v7/get_usage_report | jq .
```

```
{
  "report": "cmVwb3J0IGRhdGE=...",
  }
}
```

Types

Attribute

An Attribute represents the different values that an attribute can hold, along with the confidences of each.

- **confidence**: A float, the confidence of the value in the distribution with the highest confidence.

C#

```
float confidence = attribute.Confidence;
```

C++

```
float confidence = attribute.confidence();
```

Go

```
confidence := attribute.GetConfidence()
```

Java

```
float confidence = attribute.getConfidence();
```

JSON

```
attribute["confidence"]
```

Node

```
var confidence = attribute.getConfidence();
```

Python

```
confidence = attribute.confidence
```

Ruby

```
confidence = attribute.confidence
```

- **value**: A string, the value within the distribution with the highest confidence.

C#

```
string value = attribute.Value;
```

C++

```
string value = attribute.value();
```

Go

```
value := attribute.GetValue()
```

Java

```
string value = attribute.getValue();
```

JSON

```
attribute["value"]
```

Node

```
var value = attribute.getValue();
```

Python

```
value = attribute.value
```

Ruby

```
value = attribute.value
```

- **distribution**: A list of `AttributePairs`, the distribution of all possible values and confidences.

C#

```
var distribution = attribute.Distribution;
```

C++

```
auto distribution = attribute.distribution();
```

Go

```
distribution := attribute.GetDistribution()
```

Java

```
var distribution = attribute.getDistribution();
```

JSON

```
attribute["distribution"]
```

Node

```
var distribution = attribute.getDistributionList();
```

Python

```
distribution = attribute.distribution
```

Ruby

```
distribution = attribute.distribution
```

Attribute Pair

An Attribute Pair stores the confidence that the attribute holds a certain value. For example, an Age attribute pair with a confidence of `0.75` and value of `20-30` indicates a 75% confidence that the Face has an age between 20 and 30.

- **confidence**: A float, the confidence that the attribute holds the associated value.

C#

```
float confidence = attributePair.Confidence;
```

C++

```
float confidence = attribute_pair.confidence();
```

Go

```
confidence := attributePair.GetConfidence()
```

Java

```
float confidence = attributePair.getConfidence();
```

JSON

```
attribute_pair["confidence"]
```

Node

```
var confidence = attribute_pair.getConfidence();
```

Python

```
confidence = attribute_pair.confidence
```

Ruby

```
confidence = attribute_pair.confidence
```

- **value**: A string, the value of the attribute.

C#

```
string value = attributePair.Value;
```

C++

```
std::string value = attribute_pair.value();
```

Go

```
value := attributePair.GetValue()
```

Java

```
String value = attributePair.getValue();
```

JSON

```
attribute_pair["value"]
```

Node

```
var value = attribute_pair.getValue();
```

Python

```
value = attribute_pair.value
```

Ruby

```
value = attribute_pair.value
```

Bounding Box

A bounding box represents the rectangle around a Face.

- **origin**: A `Point`, the location of the top left corner of the rectangle.

C#

```
Point origin = boundingBox.Origin;
```

C++

```
auto origin = bounding_box.origin();
```

Go

```
origin := boundingBox.GetOrigin()
```

Java

```
Point origin = boundingBox.getOrigin();
```

JSON

```
bounding_box["origin"]
```

Node

```
var origin = bounding_box.getOrigin();
```

Python

```
origin = bounding_box.origin
```

Ruby

```
origin = bounding_box.origin
```

- **width**: A float, the width of the bounding box.

C#

```
float width = boundingBox.Width;
```

C++

```
float width = bounding_box.width();
```

Go

```
width := boundingBox.GetWidth()
```

Java

```
float width = boundingBox.getWidth();
```

JSON

```
bounding_box["width"]
```

Node

```
var width = bounding_box.getWidth();
```

Python

```
width = bounding_box.width
```

Ruby

```
width = bounding_box.width
```

- **height**: A float, the height of the bounding box.

C#

```
float height = boundingBox.Height;
```

C++

```
float height = bounding_box.height();
```

Go

```
height := boundingBox.GetHeight()
```

Java

```
float height = boundingBox.getHeight();
```

JSON

```
bounding_box["height"]
```

Node

```
var height = bounding_box.getHeight();
```

Python

```
height = bounding_box.height
```

Ruby

```
height = bounding_box.height
```

Face

A Face object, representing a Face in an image.

- **bounding_box**: A `BoundingBox`, the bounding box around the Face.

C#

```
BoundingBox boundingBox = face.BoundingBox;
```

C++

```
auto bounding_box = face.bounding_box();
```

Go

```
boundingBox := face.GetBoundingBox()
```

Java

```
BoundingBox boundingBox = face.getBoundingBox();
```

JSON

```
face["bounding_box"]
```

Node

```
var bounding_box = face.getBoundingBox();
```

Python

```
bounding_box = face.bounding_box
```

Ruby

```
bounding_box = face.bounding_box
```

- **landmarks**: A `Landmarks` object, the landmarks of the Face.

C#

```
Landmarks landmarks = face.Landmarks;
```

C++

```
auto landmarks = face.landmarks();
```

Go

```
landmarks := face.GetLandmarks()
```

Java

```
Landmarks landmarks = face.getLandmarks();
```

JSON

```
face["landmarks"]
```

Node

```
var landmarks = face.getLandmarks();
```

Python

```
landmarks = face.landmarks
```

Ruby

```
landmarks = face.landmarks
```

- **embedding**: A list of floats, the embedding of the Face which is used to compare to other faces.

C#

```
var embedding = face.Embedding;
```

C++

```
auto embedding = face.embedding();
```

Go

```
embedding := face.GetEmbedding()
```

Java

```
var embedding = face.getEmbeddingList();
```

JSON

```
face["embedding"]
```

Node

```
var embedding = face.getEmbeddingList();
```

Python

```
embedding = face.embedding
```

Ruby

```
embedding = face.embedding
```

- **ages**: An `Attribute`, the age of the Face.

C#

```
Attribute ages = face.Ages;
```

C++

```
auto ages = face.ages();
```

Go

```
ages := face.GetAges()
```

Java

```
Attribute ages = face.getAges();
```

JSON

```
face["ages"]
```

Node

```
var ages = face.getAges();
```

Python

```
ages = face.ages
```

Ruby

```
ages = face.ages
```

- **genders**: An `Attribute`, the gender of the Face.

C#

```
Attribute genders = face.Genders;
```

C++

```
auto genders = face.genders();
```

Go

```
genders := face.GetGenders()
```

Java

```
Attribute genders = face.getGenders();
```

JSON

```
face["genders"]
```

Node

```
var genders = face.getGenders();
```

Python

```
genders = face.genders
```

Ruby

```
genders = face.genders
```

- **headwear**: An `Attribute`, the headwear of the Face.

C#

```
Attribute headwear = face.Headwear;
```

C++

```
auto headwear = face.headwear();
```

Go

```
headwear := face.GetHeadwear()
```

Java

```
Attribute headwear = face.getHeadwear();
```

JSON

```
face["headwear"]
```

Node

```
var headwear = face.getHeadwear();
```

Python

```
headwear = face.headwear
```

Ruby

```
headwear = face.headwear
```

- **glasses**: An `Attribute`, the glasses of the Face.

C#

```
Attribute glasses = face.Glasses;
```

C++

```
Attribute glasses = face.glasses;
```

Go

```
glasses := face.GetGlasses()
```

Java

```
Attribute glasses = face.getGlasses();
```

JSON

```
face["glasses"]
```

Node

```
var glasses = face.getGlasses();
```

Python

```
glasses = face.glasses
```

Ruby

```
glasses = face.glasses
```

- **eyes**: An `Attribute`, the eyes of the Face.

C#

```
Attribute eyes = face.Eyes;
```

C++

```
auto eyes = face.eyes();
```

Go

```
eyes := face.GetEyes()
```

Java

```
Attribute eyes = face.getEyes();
```

JSON

```
face["eyes"]
```

Node

```
var eyes = face.getEyes();
```

Python

```
eyes = face.eyes
```

Ruby

```
eyes = face.eyes
```

- **smile**: An `Attribute`, the smile of the Face.

C#

```
Attribute smile = face.Smile
```

C++

```
auto smile = face.smile();
```

Go

```
smile := face.GetSmile()
```

Java

```
Attribute smile = face.getSmile();
```

JSON

```
face["smile"]
```

Node

```
var smile = face.getSmile();
```

Python

```
smile = face.smile
```

Ruby

```
smile = face.smile
```

- **aligned_face_image**: A bytes object, containing the cropped and prepared image of the Face required for embedding.

C#

```
var alignedFaceImage = face.AlignedFaceImage;
```

C++

```
std::string aligned_face_image = face.aligned_face_image();
```

Go

```
alignedFaceImage := face.GetAlignedFaceImage
```

Java

```
byte[] alignedFaceImage = face.getAlignedFaceImage();
```

JSON

```
face["aligned_face_image"]
```

Node

```
var aligned_face_image = face.getAlignedFaceImage();
```

Python

```
aligned_face_image = face.aligned_face_image
```

Ruby

```
aligned_face_image = face.aligned_face_image
```

- **attributes_images:** A list of bytes objects, containing the prepared images of the Face required for attributes (genders, smile, mask, headwear, glasses, eyes).

C#

```
var attrImages = face.AttributesImages;
```

C++

```
google::protobuf::RepeatedPtrField<std::string> attr_images =
face.attributes_images();
```

Go

```
attrImages := face.AttributesImages
```

Java

```
Iterable<ByteString> attrImages = face.getAttributesImagesList();
```

JSON

```
face["attributes_images"]
```

Node

```
var attr_images = face.getAttributesImagesList();
```

Python

```
attr_images = face.attributes_images
```

Ruby

```
attr_images = face.attributes_images
```

- **deepfake_images:** A list of bytes objects, containing the prepared images of the Face required for deepfake.

C#

```
var deepfakeImages = face.DeepfakeImages;
```

C++

```
google::protobuf::RepeatedPtrField<std::string> deepfake_images =
face.deepfake_images();
```

Go

```
deepfakeImages := face.DeepfakeImages
```

Java

```
Iterable<ByteString> deepfakeImages = face.getDeepfakeImagesList();
```

JSON

```
face["deepfake_images"]
```

Node

```
var deepfake_images = face.getDeepfakeImagesList();
```

Python

```
deepfake_images = face.deepfake_images
```

Ruby

```
deepfake_images = face.deepfake_images
```

- **liveness_image:** A bytes object, containing the prepared images of the Face required for liveness.

C#

```
var livenessImage = face.LivenessImage;
```

C++

```
std::string liveness_image = face.liveness_image();
```

Go

```
livenessImage := face.LivenessImage
```

Java

```
byte[] livenessImage = face.getLivenessImage();
```

JSON

```
face["liveness_image"]
```

Node

```
var liveness_image = face.getLivenessImage();
```

Python

```
liveness_image = face.liveness_image
```

Ruby

```
liveness_image = face.liveness_image
```

- **ages_v2_image:** A bytes object, containing the prepared images of the Face required for ages_v2.

C#

```
var agesV2Image = face.AgesV2Image;
```

C++

```
std::string ages_v2_image = face.ages_v2_image();
```

Go

```
agesV2Image := face.AgesV2Image
```

Java

```
byte[] agesV2Image = face.getAgesV2Image();
```

JSON

```
face["ages_v2_image"]
```

Node

```
var ages_v2_image = face.getAgesV2Image();
```

Python

```
ages_v2_image = face.ages_v2_image
```

Ruby

```
ages_v2_image = face.ages_v2_image
```

- **embedding_template**: A bytes object, containing the embedding as a binary blob.

C#

```
var embeddingTemplate = face.EmbeddingTemplate
```

C++

```
std::string embedding_template = face.embedding_template();
```

Go

```
embeddingTemplate := face.GetEmbeddingTemplate()
```

Java

```
var embeddingTemplate = face.getEmbeddingTemplate();
```

JSON

```
face["embedding_template"]
```

Node

```
var embedding_template = face.getEmbeddingTemplate();
```

Python

```
embedding_template = face.embedding_template
```

Ruby

```
embedding_template = face.embedding_template
```

- **liveness_validness**: A `Validness` object, the validness result for liveness.

C#

```
var livenessValidness = face.LivenessValidness
```

C++

```
auto liveness_validness = face.liveness_validness();
```

Go

```
livenessValidness := face.GetLivenessValidness()
```

Java

```
var livenessValidness = face.getLivenessValidness();
```

JSON

```
face["liveness_validness"]
```

Node

```
var liveness_validness = face.getLivenesValidness();
```

Python

```
liveness_validness = face.liveness_validness
```

Ruby

```
liveness_validness = face.liveness_validness
```

- **liveness**: A `Liveness` object, the liveness result.

C#

```
var liveness = face.Liveness
```

C++

```
auto liveness = face.liveness();
```

Go

```
liveness := face.GetLiveness()
```

Java

```
var liveness = face.getLiveness();
```

JSON

```
face["liveness"]
```

Node

```
var liveness = face.getLiveness();
```

Python

```
liveness = face.liveness
```

Ruby

```
liveness = face.liveness
```

- **ages_v2_validness:** A `Validness` object, the validness result for ages_v2.

C#

```
var ageValidness = face.AgesV2Validness
```

C++

```
auto age_validness = face.ages_v2_validness();
```

Go

```
ageValidness := face.AgesV2Validness
```

Java

```
var ageValidness = face.getAgesV2Validness();
```

JSON

```
face["ages_v2_validness"]
```

Node

```
var age_validness = face.getAgesV2Validness();
```

Python

```
age_validness = face.ages_v2_validness
```

Ruby

```
age_validness = face.ages_v2_validness
```

- **ages_v2**: A float, the age of the face in a given image.

C#

```
var age = face.AgesV2
```

C++

```
auto age = face.ages_v2();
```

Go

```
age := face.AgesV2
```

Java

```
var age = face.getAgesV2();
```

JSON

```
face["ages_v2"]
```

Node

```
var age = face.getAgesV2();
```

Python

```
age = face.ages_v2
```

Ruby

```
age = face.ages_v2
```

- **acceptability**: A float between 0 and 1, measuring likelihood of a real human face in the bounding box.

C#

```
float acceptability = attribute.Acceptability;
```

C++

```
float acceptability = attribute.acceptability();
```

Go

```
acceptability := attribute.GetAcceptability()
```

Java

```
float acceptability = attribute.getAcceptability();
```

JSON

```
face["acceptability"]
```

Node

```
var acceptability = attribute.getAcceptability();
```

Python

```
acceptability = attribute.acceptability
```

Ruby

```
acceptability = attribute.acceptability
```

- **quality**: A float between 0 and 1, measuring how good the Face is for recognition.

C#

```
float quality = attribute.Quality;
```

C++

```
float quality = attribute.quality();
```

Go

```
quality := attribute.GetQuality()
```

Java

```
float quality = attribute.getQuality();
```

JSON

```
face["quality"]
```

Node

```
var quality = attribute.getQuality();
```

Python

```
quality = attribute.quality
```

Ruby

```
quality = attribute.quality
```

- **mask**: A float between 0 and 1, representing the probability of the Face wearing mask.

C#

```
float mask = attribute.Mask;
```

C++

```
float mask = attribute.mask();
```

Go

```
mask := attribute.GetMask()
```

Java

```
float mask = attribute.getMask();
```

JSON

```
face["mask"]
```

Node

```
var mask = attribute.getMask();
```

Python

```
mask = attribute.mask
```

Ruby

```
mask = attribute.mask
```

- **deepfake_validness:** A `Validness` object, the validness result for deepfake.

C#

```
var validness = face.DeepfakeValidness
```

C++

```
auto validness = face.deepfake_validness();
```

Go

```
validness := face.DeepfakeValidness
```

Java

```
var validness = face.getDeepfakeValidness();
```

JSON

```
face["deepfake_validness"]
```

Node

```
var validness = face.getDeepfakeValidness();
```

Python

```
validness = face.deepfake_validness
```

Ruby

```
validness = face.deepfake_validness
```

- **deepfake_realness**: A float between 0 and 1, representing the probability of an unaltered Face image.

⚠ NOTE: This is now deprecated. Please use the Deepfake object instead.

C#

```
float dfRealness = attribute.DeepfakeRealness;
```

C++

```
float df_realness = attribute.deepfake_realness();
```

Go

```
dfRealness := attribute.DeepfakeRealness
```

Java

```
float dfRealness = attribute.getDeepfakeRealness();
```

JSON

```
face["deepfake_realness"]
```

Node

```
var dfRealness = attribute.getDeepfakeRealness();
```

Python

```
df_realness = attribute.deepfake_realness
```

Ruby

```
df_realness = attribute.deepfake_realness
```

Validness

- **sharpness**: A float between 0 and 1, representing the sharpness of the face in an image.
- **quality**: A float between 0 and 1, representing the quality of the face in an image.
- **acceptability**: A float between 0 and 1, representing the acceptability of the face in an image.
- **frontality**: A float between 0 and 1, representing the frontality of the face.
- **image_average_pixel_value**: A float representing the average of all face pixels in a grey-scaled face image.
- **image_bright_pixel_pct**: A float representing the percentage of face pixels that are over saturated (too bright) in a RGB face image. Returned for `LIVENESS_VALIDNESS` only.
- **image_dark_pixel_pct**: A float representing the percentage of face pixels that are under saturated (too dark) in a RGB face image. Returned for `LIVENESS_VALIDNESS` only.

- **mask_probability**: A float between 0 and 1, representing the probability of wearing a mask.
- **face_width_pct**: A float representing the width-percentage of the image the face occupies.
- **face_height_pct**: A float representing the height-percentage of the image the face occupies.
- **face_width**: A float representing the width of the face in pixels.
- **face_height**: A float representing the height of the face in pixels.
- **position_pcts**: A list of floats representing the positions on the face.
- **face_roll_angle:** A float representing an absolute angle in degrees the face may be rotated around its forward axis.
- **is_valid**: a boolean representing if the image is valid.
- **feedback**: A list of Feedback representing issues found with the image.

Feedback

An enum type providing feedback on why validness failed.

```
UNKNOWN - An unknown error occured, check the server logs
FACE_QUALITY_POOR - The face has poor quality
FACE_ACCEPTABILITY_POOR - The face has poor acceptability
IMG_LIGHTING_DARK - The image has low light
IMG_LIGHTING_BRIGHT - The image is too bright
FACE_SIZE_LARGE - The face is too large / too close to the camera.
FACE_POS_LEFT - The face is too far to the left of the image.
FACE_POS_RIGHT - The face is too far to the right of the image.
FACE_POS_HIGH - The face is too close to the top of the image.
FACE_POS_LOW - The face is too close to the bottom of the image.
FACE_MASK_FOUND - The face has a mask on.
FACE_FRONTALITY_POOR - The face is not facing frontward.
FACE_SHARPNESS_POOR - The face has low sharpness.
FACE_NOT_FOUND - No faces were found.
TOO_MANY_FACES - Too many faces were found.
FACE_RES_LOW - The image has low resolution.
FACE_ROLL_ANGLE_EXCEEDED - The face is rotated around its forward axis.
OTHER - An error occured, check the server logs.
```

Liveness

- **liveness_probability**: A float between 0 and 1, representing he probability of the face being considered real and alive.

C#

```
float liveness_probability = liveness.LivenessProbability;
```

C++

```
float liveness_probability = liveness.liveness_probability();
```

Go

```
livenessProbability := liveness.GetLivenessProbability()
```

Java

```
Point leftEye = landmarks.getLivenessProbability();
```

JSON

```
liveness["liveness_probability"]
```

Node

```
var liveness_probability = liveness.getLivenessProbability();
```

Python

```
liveness_probability = liveness.liveness_probability
```

Ruby

```
liveness_probability = liveness.liveness_probability
```

Deepfake

- realness: A float between 0 and 1, representing the probability of the face being considered real, authentic, and unaltered.

C#

```
float realness = deepfake.Realness;
```

C++

```
float realness = deepfake.realness();
```

Go

```
realness := deepfake.GetRealness()
```

Java

```
float realness = deepfake.getRealness();
```

JSON

```
deepfake["realness"]
```

Node

```
var realness = deepfake.getRealness();
```

Python

```
realness = deepfake.realness
```

Ruby

```
realness = deepfake.realness
```

- type: The type of deepfake detected (e.g. face_swap, synthetic).

C#

```
float type = deepfake.Type;
```

C++

```
float deepfake_type = deepfake.type();
```

Go

```
deepfakeType := deepfake.GetType()
```

Java

```
float deepfakeType = deepfake.getType();
```

JSON

```
deepfake["type"]
```

Node

```
var deepfakeType = deepfake.getType();
```

Python

```
deepfake_type = deepfake.type
```

Ruby

```
deepfake_type = deepfake.type
```

- detailed_scores: The probability of each deepfake type for a given face.

C#

```
float x = deepfake.DetailedScores.FaceSwapX;
float y = deepfake.DetailedScores.FaceSwapY;
float syn = deepfake.DetailedScores.Synthetic;
```

C++

```
float x = deepfake.detailed_scores().face_swap_x();
float y = deepfake.detailed_scores().face_swap_y();
float syn = deepfake.detailed_scores().synthetic();
```

Go

```
x := deepfake.GetDetailedScores().GetFaceSwapX();
y := deepfake.GetDetailedScores().GetFaceSwapY();
syn := deepfake.GetDetailedScores().GetSynthetic();
```

Java

```
float x = deepfake.getDetailedScores().getFaceSwapX();
float y = deepfake.getDetailedScores().getFaceSwapY();
float syn = deepfake.getDetailedScores().getSynthetic();
```

JSON

```
deepfake["detailed_scores"]["face_swap_x"]
deepfake["detailed_scores"]["face_swap_y"]
deepfake["detailed_scores"]["synthetic"]
```

Node

```
var x = deepfake.getDetailedScores().getFaceSwapX();
var y = deepfake.getDetailedScores().getFaceSwapY();
var syn = deepfake.getDetailedScores().getSynthetic();
```

Python

```
x = deepfake.detailed_scores.face_swap_x;
y = deepfake.detailed_scores.face_swap_y;
syn = deepfake.detailed_scores.synthetic;
```

Ruby

```
x = deepfake.detailed_scores.face_swap_x;
y = deepfake.detailed_scores.face_swap_y;
syn = deepfake.detailed_scores.synthetic;
```

Landmarks

A Landmarks object represents the landmarks on a Face.

- **left_eye**: A `Point`, the location of the Face's left eye.

C#

```
Point leftEye = landmarks.LeftEye;
```

C++

```
auto left_eye = landmarks.left_eye();
```

Go

```
leftEye := landmarks.GetLeftEye()
```

Java

```
Point leftEye = landmarks.getLeftEye();
```

JSON

```
landmarks["left_eye"]
```

Node

```
var left_eye = landmarks.getLeftEye();
```

Python

```
left_eye = landmarks.left_eye
```

Ruby

```
left_eye = landmarks.left_eye
```

- **right_eye**: A `Point`, the location of the Face's right eye.

C#

```
Point rightEye = landmarks.RightEye;
```

C++

```
auto right_eye = landmarks.right_eye();
```

Go

```
rightEye := landmarks.GetRightEye()
```

Java

```
Point rightEye = landmarks.getRightEye();
```

JSON

```
landmarks["right_eye"]
```

Node

```
var right_eye = landmarks.getRightEye();
```

Python

```
right_eye = landmarks.right_eye
```

Ruby

```
right_eye = landmarks.right_eye
```

- **nose**: A `Point`, the location of the Face's nose.

C#

```
Point nose = landmarks.Nose;
```

C++

```
auto nose = landmarks.nose();
```

Go

```
nose := landmarks.GetNose()
```

Java

```
Point nose = landmarks.getNose();
```

JSON

```
landmarks["nose"]
```

Node

```
var nose = landmarks.getNose();
```

Python

```
nose = landmarks.nose
```

Ruby

```
nose = landmarks.nose
```

- **left_mouth**: A `Point`, the location of the left edge of the Face's mouth.

C#

```
Point leftMouth = landmarks.LeftMouth;
```

C++

```
auto left_mouth = landmarks.left_mouth();
```

Go

```
leftMouth := landmarks.GetLeftMouth()
```

Java

```
Point leftMouth = landmarks.getLeftMouth();
```

JSON

```
landmarks["left_mouth"]
```

Node

```
var left_mouth = landmarks.getLeftMouth();
```

Python

```
left_mouth = landmarks.left_mouth
```

Ruby

```
left_mouth = landmarks.left_mouth
```

- **right_mouth**: A `Point`, the location of the right edge of the Face's mouth.

C#

```
Point rightMouth = landmarks.RightMouth;
```

C++

```
auto right_mouth = landmarks.right_mouth();
```

Go

```
rightMouth := landmarks.GetRightMouth()
```

Java

```
Point rightMouth = landmarks.getRightMouth();
```

JSON

```
landmarks["right_mouth"]
```

Node

```
var right_mouth = landmarks.getRightMouth();
```

Python

```
right_mouth = landmarks.right_mouth
```

Ruby

```
right_mouth = landmarks.right_mouth
```

Metadata Response

A MetadataResponse object stores useful parameters from Processor API server.

- **recognition_model_name**: A string, the name of the Recognition model used by the running server instance.

C#

```
string recognitionModelName = metadata.RecognitionModelName;
```

C++

```
std::string recognition_model_name =
metadata.recognition_model_name();
```

Go

```
recognitionModelName := metadata.GetRecognitionModelName()
```

Java

```
string recognitionModelName = metadata.getRecognitionModelName();
```

JSON

```
response["recognition_model_name"]
```

Node

```
var recognition_model_name = metadata.getRecognitionModelName();
```

Python

```
recognition_model_name = metadata.recognition_model_name
```

Ruby

```
recognition_model_name = metadata.recognition_model_name
```

- **engine_name**: A string, the name of the engine used by the running server instance.

C#

```
string engineName = metadata.EngineName;
```

C++

```
std::string engine_name = metadata.engine_name();
```

Go

```
engineName := metadata.GetEngineName()
```

Java

```
string engineName = metadata.getEngineName();
```

JSON

```
response["engine_name"]
```

Node

```
var engine_name = metadata.getEngineName();
```

Python

```
engine_name = metadata.engine_name
```

Ruby

```
engine_name = metadata.engine_name
```

- **recognition_sdk_version**: A string, Recognition SDK version used by the running server instance.

C#

```
string recognitionSdkVersion = metadata.RecognitionSdkVersion;
```

C++

```
std::string recognition_sdk_version =
metadata.recognition_sdk_version();
```

Go

```
recognitionSdkVersion := metadata.GetRecognitionSdkVersion()
```

Java

```
string recognitionSdkVersion = metadata.getRecognitionSdkVersion();
```

JSON

```
response["recognition_sdk_version"]
```

Node

```
var recognition_sdk_version = metadata.getRecognitionSdkVersion();
```

Python

```
recognition_sdk_version = metadata.recognition_sdk_version
```

Ruby

```
recognition_sdk_version = metadata.recognition_sdk_version
```

- **recognition_model_version**: A string, Recognition model version used by the running server instance.

C#

```
string recognitionModelVersion = metadata.RecognitionModelVersion;
```

C++

```
std::string recognition_model_version =
metadata.recognition_model_version();
```

Go

```
recognitionModelVersion := metadata.GetRecognitionModelVersion()
```

Java

```
string recognitionModelVersion =
metadata.getRecognitionModelVersion();
```

JSON

```
response["recognition_model_version"]
```

Node

```
var recognition_model_version =
metadata.getRecognitionModelVersion();
```

Python

```
recognition_model_version = metadata.recognition_model_version
```

Ruby

```
recognition_model_version = metadata.recognition_model_version
```

- **attributes_model_name**: A string, the name of the Recognition model used by the running server instance.

C#

```
string attributesModelName = metadata.AttributesModelName;
```

C++

```
std::string attributes_model_name =
metadata.attributes_model_name();
```

Go

```
attributesModelName := metadata.GetAttributesModelName()
```

Java

```
string attributesModelName = metadata.getAttributesModelName();
```

JSON

```
response["attributes_model_name"]
```

Node

```
var attributes_model_name = metadata.getAttributesModelName();
```

Python

```
attributes_model_name = metadata.attributes_model_name
```

Ruby

```
attributes_model_name = metadata.attributes_model_name
```

- **attributes_sdk_version**: A string, Recognition SDK version used by the running server instance.

C#

```
string attributesSdkVersion = metadata.AttributesSdkVersion;
```

C++

```
std::string attributes_sdk_version =
metadata.attributes_sdk_version();
```

Go

```
attributesSdkVersion := metadata.GetAttributesSdkVersion()
```

Java

```
string attributesSdkVersion = metadata.getAttributesSdkVersion();
```

JSON

```
response["attributes_sdk_version"]
```

Node

```
var attributes_sdk_version = metadata.getAttributesSdkVersion();
```

Python

```
attributes_sdk_version = metadata.attributes_sdk_version
```

Ruby

```
attributes_sdk_version = metadata.attributes_sdk_version
```

- **attributes_model_version**: A string, Attributes model version used by the running server instance.

C#

```
string attributesModelVersion = metadata.AttributesModelVersion;
```

C++

```
std::string attributes_model_version =
metadata.attributes_model_version();
```

Go

```
attributesModelVersion := metadata.GetAttributesModelVersion()
```

Java

```
string attributesModelVersion =
metadata.getAttributesModelVersion();
```

JSON

```
response["attributes_model_version"]
```

Node

```
var attributes_model_version = metadata.getAttributesModelVersion();
```

Python

```
attributes_model_version = metadata.attributes_model_version
```

Ruby

```
attributes_model_version = metadata.attributes_model_version
```

- **liveness2d_model_name**: A string, the name of the Liveness2d model used by the running server instance.

C#

```
string liveness2dModelName = metadata.Liveness2dModelName;
```

C++

```
std::string liveness2d_model_name =
metadata.liveness2d_model_name();
```

Go

```
liveness2dModelName := metadata.GetLiveness2dModelName()
```

Java

```
string liveness2dModelName = metadata.getLiveness2dModelName();
```

JSON

```
response["liveness2d_model_name"]
```

Node

```
var liveness2d_model_name = metadata.getLiveness2dModelName();
```

Python

```
liveness2d_model_name = metadata.liveness2d_model_name
```

Ruby

```
liveness2d_model_name = metadata.liveness2d_model_name
```

- **liveness2d_sdk_version**: A string, Liveness2d SDK version used by the running server instance.

C#

```
string liveness2dSdkVersion = metadata.Liveness2dSdkVersion;
```

C++

```
std::string liveness2d_sdk_version =
metadata.liveness2d_sdk_version();
```

Go

```
liveness2dSdkVersion := metadata.GetLiveness2dSdkVersion()
```

Java

```
string liveness2dSdkVersion = metadata.getLiveness2dSdkVersion();
```

JSON

```
response["liveness2d_sdk_version"]
```

Node

```
var liveness2d_sdk_version = metadata.getLiveness2dSdkVersion();
```

Python

```
liveness2d_sdk_version = metadata.liveness2d_sdk_version
```

Ruby

```
liveness2d_sdk_version = metadata.liveness2d_sdk_version
```

- **liveness2d_model_version**: A string, Liveness2d model version used by the running server instance.

C#

```
string liveness2dModelVersion = metadata.Liveness2dModelVersion;
```

C++

```
std::string liveness2d_model_version =
metadata.attributes_model_version();
```

Go

```
liveness2dModelVersion := metadata.Getliveness2dModelVersion()
```

Java

```
string liveness2dModelVersion =
metadata.getLiveness2dModelVersion();
```

JSON

```
response["liveness2d_model_version"]
```

Node

```
var liveness2d_model_version = metadata.getLiveness2dModelVersion();
```

Python

```
liveness2d_model_version = metadata.liveness2d_model_version
```

Ruby

```
liveness2d_model_version = metadata.liveness2d_model_version
```

- **deepfake_model_name**: A string, the name of the Deepfake model used by the running server instance.

C#

```
string liveness2dModelName = metadata.Liveness2dModelName;
```

C++

```
std::string deepfake_model_name = metadata.deepfake_model_name();
```

Go

```
deepfakeModelName := metadata.GetDeepfakeModelName()
```

Java

```
string deepfakeModelName = metadata.getDeepfakeModelName();
```

JSON

```
response["deepfake_model_name"]
```

Node

```
var deepfkae_model_name = metadata.getDeepfakeModelName();
```

Python

```
deepfake_model_name = metadata.deepfkae_model_name
```

Ruby

```
deepfake_model_name = metadata.deepfake_model_name
```

- **deepfake_model_version**: A string, Deepfake model version used by the running server instance.

C#

```
string deepfkaeModelVersion = metadata.DeepfakeModelVersion;
```

C++

```
std::string deepfake_model_version =
metadata.deepfake_model_version();
```

Go

```
deepfakeModelVersion := metadata.GetDeepfakeModelVersion()
```

Java

```
string deepfakeModelVersion = metadata.getDeepfakeModelVersion();
```

JSON

```
response["deepfake_model_version"]
```

Node

```
var deepfkae_model_version = metadata.getDeepfakeModelVersion();
```

Python

```
deepfake_model_version = metadata.deepfake_model_version
```

Ruby

```
deepfake_model_version = metadata.deepfake_model_version
```

- **deepfake_sdk_version**: A string, Deepfake SDK version used by the running server instance.

C#

```
string deepfakeSdkVersion = metadata.DeepfakeSdkVersion;
```

C++

```
std::string deepfake_sdk_version = metadata.deepfake_sdk_version();
```

Go

```
deepfakeSdkVersion := metadata.GetDeepfakeSdkVersion()
```

Java

```
string deepfakeSdkVersion = metadata.getDeepfakeSdkVersion();
```

JSON

```
response["deepfake_sdk_version"]
```

Node

```
var deepfake_sdk_version = metadata.getDeepfakeSdkVersion();
```

Python

```
deepfake_sdk_version = metadata.deepfake_sdk_version
```

Ruby

```
depfake_sdk_version = metadata.deepfake_sdk_version
```

- **ages_v2_model_name**: A string, the name of the Age model used by the running server instance.

C#

```
string modelName = metadata.AgesV2ModelName;
```

C++

```
std::string model_name = metadata.ages_v2_model_name();
```

Go

```
modelName := metadata.GetAgesV2ModelName()
```

Java

```
string modelName = metadata.getAgesV2ModelName();
```

JSON

```
response["ages_v2_model_name"]
```

Node

```
var model_name = metadata.getAgesV2ModelName();
```

Python

```
model_name = metadata.ages_v2_model_name
```

Ruby

```
model_name = metadata.ages_v2_model_name
```

- **ages_v2_model_version**: A string, Age model version used by the running server instance.

C#

```
string modelVersion = metadata.AgesV2ModelVersion;
```

C++

```
std::string model_version = metadata.ages_v2_model_version();
```

Go

```
modelVersion := metadata.GetAgesV2ModelVersion()
```

Java

```
string modelVersion = metadata.getAgesV2ModelVersion();
```

JSON

```
response["ages_v2_model_version"]
```

Node

```
var model_version = metadata.getAgesV2ModelVersion();
```

Python

```
model_version = metadata.ages_v2_model_version
```

Ruby

```
model_version = metadata.ages_v2_model_version
```

- **ages_v2_sdk_version**: A string, Age SDK version used by the running server instance.

C#

```
string sdkVersion = metadata.AgesV2SdkVersion;
```

C++

```
std::string sdk_version = metadata.ages_v2_sdk_version();
```

Go

```
string sdkVersion = metadata.getAgesV2SdkVersion();
```

Java

```
string sdkVersion = metadata.getAgesV2SdkVersion();
```

JSON

```
response["ages_v2_sdk_version"]
```

Node

```
var sdk_version = metadata.getAgesV2SdkVersion();
```

Python

```
sdk_version = metadata.ages_v2_sdk_version
```

Ruby

```
sdk_version = metadata.ages_v2_sdk_version
```

- **standard_scoring_parameters**: A `ScoringParameters` object, holds parameters for `standard` scoring mode for use by embeddings comparison functions.

C#

```
var parameters = metadata.StandardScoringParameters;
```

C++

```
auto parameters = metadata.standard_scoring_parameters();
```

Go

```
parameters := metadata.GetStandardScoringParameters()
```

Java

```
ScoringParameters parameters =
metadata.getStandardScoringParameters();
```

JSON

```
response["standard_scoring_parameters"]
```

Node

```
var parameters = metadata.getStandardScoringParameters();
```

Python

```
parameters = metadata.standard_scoring_parameters
```

Ruby

```
parameters = metadata.standard_scoring_parameters
```

- **enhanced_scoring_parameters**: A `ScoringParameters` object, holds parameters for `enhanced` scoring mode for use by embeddings comparison functions.

C#

```
var parameters = metadata.EnhancedScoringParameters;
```

C++

```
auto parameters = metadata.enhanced_scoring_parameters();
```

Go

```
parameters := metadata.GetEnhancedScoringParameters()
```

Java

```
ScoringParameters parameters =
metadata.getEnhancedScoringParameters();
```

JSON

```
response["enhanced_scoring_parameters"]
```

Node

```
var parameters = metadata.getEnhancedScoringParameters();
```

Python

```
parameters = metadata.enhanced_scoring_parameters
```

Ruby

```
parameters = metadata.enhanced_scoring_parameters
```

- **app_version**: A string, server version.

C#

```
string appVersion = metadata.AppVersion;
```

C++

```
std::string app_version = metadata.app_version();
```

Go

```
appVersion := metadata.GetAppVersion()
```

Java

```
string appVersion = metadata.getAppVersion();
```

JSON

```
response["app_version"]
```

Node

```
var app_version = metadata.getAppVersion();
```

Python

```
app_version = metadata.app_version
```

Ruby

```
app_version = metadata.app_version
```

- **liveness_default_threshold**: A Liveness Validness Thresholds, holds default thresholds for liveness validness.

C#

```
var thresholds = metadata.LivenessDefaultThresholds;
```

C++

```
LivenessValidnessThresholds thresholds =
metadata.liveness_default_thresholds();
```

Go

```
thresholds := metadata.GetLivenessDefaultThresholds()
```

Java

```
LivenessValidnessThresholds thresholds =
metadata.getLivenessDefaultThresholds();
```

JSON

```
response["liveness_default_thresholds"]
```

Node

```
let thresholds = metadata.getLivenessDefaultThresholds();
```

Python

```
thresholds = metadata.liveness_default_thresholds
```

Ruby

```
thresholds = metadata.liveness_default_thresholds
```

- **ages_v2_default_threshold**: A Ages_V2 Validness Thresholds, holds default thresholds for ages_v2 validness.

C#

```
var thresholds = metadata.AgesV2DefaultThresholds;
```

C++

```
AgesV2ValidnessThresholds thresholds =
metadata.ages_v2_default_thresholds();
```

Go

```
thresholds := metadata.GetAgesV2DefaultThresholds()
```

Java

```
AgesV2ValidnessThresholds thresholds =
metadata.getAgesV2DefaultThresholds();
```

JSON

```
response["ages_v2_default_thresholds"]
```

Node

```
let thresholds = metadata.getAgesV2DefaultThresholds();
```

Python

```
thresholds = metadata.ages_v2_default_thresholds
```

Ruby

```
thresholds = metadata.ages_v2_default_thresholds
```

- **deepfake_default_threshold**: A Deepfake Validness Thresholds, holds default thresholds for deepfake validness.

C#

```
var thresholds = metadata.DeepfakeDefaultThresholds;
```

C++

```
DeepfakeValidnessThresholds thresholds =
metadata.deepfake_default_thresholds();
```

Go

```
thresholds := metadata.GetDeepfakeDefaultThresholds()
```

Java

```
DeepfakeValidnessThresholds thresholds =
metadata.getDeepfakeDefaultThresholds();
```

JSON

```
response["deepfake_default_thresholds"]
```

Node

```
let thresholds = metadata.getDeepfakeDefaultThresholds();
```

Python

```
thresholds = metadata.deepfake_default_thresholds
```

Ruby

```
thresholds = metadata.deepfake_default_thresholds
```

Point

A Point objects represents a single point on an image.

- **x**: A float, the x value of the point.

C#

```
float x = point.X;
```

C++

```
float x = point.x();
```

Go

```
x := point.GetX()
```

Java

```
float x = point.getX();
```

JSON

```
point["x"]
```

Node

```
var x = point.getX();
```

Python

```
x = point.x
```

Ruby

```
x = point.x
```

- **y**: A float, the y value of the point.

C#

```
float y = point.Y;
```

C++

```
float y = point.y();
```

Go

```
y := point.GetY()
```

Java

```
float y = point.getY();
```

JSON

```
point["y"]
```

Node

```
var y = point.getY();
```

Python

```
y = point.y
```

Ruby

```
y = point.y
```

UsageReport Response

A UsageReportResponse object contains an encoded log of features transacted by the Processor API.

C#

```
var report = metadata.report;
```

C++

```
std::string report = response.report();
```

Go

```
report := response.GetReport()
```

Java

```
String report = metadata.getReport();
```

JSON

```
response["report"]
```

Node

```
let report = response.getReport();
```

Python

```
report = response.report
```

Ruby

```
report = response.report
```

Image Source

An enum providing the list of options to pass into `Process Full Image` function.

- **UNKNOWN**
- **MOBILE**
- **WEBCAM**

Different face detection models are selected at runtime based on the value of the environment variable `PV_DETECTION_MODEL` and `image_source` set in the `Process Full Image` request, as described below.

PV_DETECTION_MODEL	image_source	Face detection model
default	UNKNOWN	default
default	WEBCAM	mobile
default	MOBILE	mobile
mobile	UNKNOWN	mobile
mobile	WEBCAM	mobile
mobile	MOBILE	mobile
gen6.1	UNKNOWN	gen6.1
gen6.1	WEBCAM	mobile
gen6.1	MOBILE	mobile

1. For Liveness feature, it is crucial to specify the correct image source to achiever higher accuracy.
2. In Liveness validness, different thresholds are used for `min_size_size` validness parameters based on the image source. Refer docs for more details.

Process Prepared Face Image Options

An enum providing the list of options to pass into the `Process Prepared Face Image` function. Below are the possible values.

- **EMBEDDING**

- **AGES**
- **GENDERS**
- **MASK**
- **HEADWEAR**
- **GLASSES**
- **EYES**
- **SMILE**
- **AGES_V2**
- **DEEPFAKE**
- **LIVENESS**
- **ADVANCED_DATA**

Process Full Image Options

An enum providing the list of options to pass into the `Process Full Image` function. Below are the possible values.

- **BOUNDING_BOX**
- **LANDMARKS**
- **ALIGNED_FACE_IMAGE**
- **ATTRIBUTES_IMAGES**
- **QUALITY**
- **EMBEDDING**
- **AGES**
- **GENDERS**
- **MASK**
- **HEADWEAR**
- **GLASSES**
- **EYES**
- **SMILE**

- **LIVENESS**
- **LIVENESS_VALIDNESS**
- **LIVENESS_IMAGE**
- **DEEPFAKE**
- **DEEPFAKE_VALIDNESS**
- **DEEPFAKE_IMAGES**
- **AGES_V2**
- **AGES_V2_VALIDNESS**
- **AGES_V2_IMAGE**
- **ADVANCED_DATA**

Process Full Image Liveness Validness Parameters

An struct providing the validness parameters to pass into the `Process Full Image` function for liveness inference. The values in the following code block are the defaults when image source is `unknown` or `webcam`, you do not need to provide them.

C#

```
var livenessValidnessParameters = new
ProcessFullImageLivenessValidnessParamaters {
    MinFaceSharpness = 0.15,
    MinFaceQuality = 0.5,
    MinFaceAcceptability = 0.15,
    MinFaceFrontality = 70,
    MaxFaceMaskProbibility = 0.5,
    ImageIlluminationControl = 50,
    MaxFaceSizePct = 0.72,
    ImageBoundaryWidthPct = 0.8,
    ImageBoundaryHeightPct = 0.8,
    MaxFaceRollAngle = 45,
    MinFaceSize = 100,
    FailFast = true
};
```

C++

```
auto livenss_validness_parameters =
ProcessFullImageLivenessValidnessParameters{};

livenss_validness_parameters.set_min_face_sharpness(0.15);
livenss_validness_parameters.set_min_face_quality(0.5);
livenss_validness_parameters.set_min_face_acceptability(0.15);
livenss_validness_parameters.set_min_face_frontality(70);
livenss_validness_parameters.set_max_face_mask_probability(0.5);
livenss_validness_parameters.set_image_illumination_control(50);
livenss_validness_parameters.set_max_face_size_pct(0.72);
livenss_validness_parameters.set_image_boundary_width_pct(0.8);
livenss_validness_parameters.set_image_boundary_height_pct(0.8);
livenss_validness_parameters.set_min_face_size(100);
livenss_validness_parameters.set_max_face_roll_angle(45);
livenss_validness_parameters.set_fail_fast(true);
```

Go

```
livenessValidnessParameters :=
&pb.ProcessFullImageRequest_LivenessValidnessParameters{
    MinFaceSharpness:         0.15,
    MinFaceQuality:           0.5,
    MinFaceAcceptability:     0.15,
    MinFaceFrontality:        70,
    MaxFaceMaskProbibility:   0.5,
    ImageIlluminationControl: 50,
    MaxFaceSizetPct:          0.72,
    ImageBoundaryWidthPct:    0.8,
    ImageBoundaryHeightPct:   0.8,
    MinFaceSize:              100,
    MaxFaceRollAngle:         45,
    FailFast:                 true,
}
```

Java

```
ProcessFullImageRequest.LivenessValidnessParameters params =
ProcessFullImageRequest
    .LivenessValidnessParameters
    .newBuilder()
        .setMinFaceSharpness(0.15)
        .setMinFaceQuality(0.5)
        .setMinFaceAcceptability(0.15)
        .setMinFaceFrontality(70)
        .setMaxFaceMaskProbibility(0.5)
        .setImageIlluminationControl(50)
        .setMaxFaceSizePct(0.72)
        .setImageBoundaryWidthPct(0.8)
        .setImageBoundardHeight(0.8)
        .setMinFaceSize(100)
        .setMaxFaceRollAngle(45)
        .setFailFast(true)
        .build();
```

JSON

```
{
    "min_face_sharpness": 0.15,
    "min_face_quality": 0.5,
    "min_face_acceptability": 0.15,
    "min_face_frontality": 70,
    "max_face_mask_probability": 0.5,
    "image_illumination_control": 50,
    "max_face_size_pct": 0.72,
    "image_boundary_width_pct": 0.8,
    "image_boundary_height_pct": 0.8,
    "min_face_size": 100,
    "max_face_roll_angle": 45,
    "fail_fast": true
}
```

Node

```
let livenessValidnessParameters = new
ProcessFullImageLivenessValidnessParameters()
    .setMinFaceSharpness(0.15)
    .setMinFaceQuality(0.5)
    .setMinFaceAcceptability(0.15)
    .setMinFaceFrontality(70)
    .setMaxFaceMaskProbibility(0.5)
    .setImageIlluminationControl(50)
    .setMaxFaceSizePct(0.72)
    .setImageBoundaryWidthPct(0.8)
    .setImageBoundardHeight(0.8)
    .setMinFaceSize(100)
    .setMaxFaceRollAngle(45)
    .setFailFast(true);
```

Python

```
liveness_validness_parameters =
ProcessFullImageLivenessValidnessParameters(
    min_face_sharpness=0.15,
    min_face_quality=0.5,
    min_face_acceptability=0.15,
    min_face_frontality=70,
    max_face_mask_probability=0.5,
    image_illumination_control=50,
    max_face_size_pct=0.72,
    image_boundary_width_pct=0.8,
    image_boundary_height_pct=0.8,
    min_face_size=100,
    max_face_roll_angle=45,
    fail_fast=True,
)
```

Ruby

```
liveness_validness_parameters =
ProcessFullImageLivenessValidnessParameters.new(
    min_face_sharpness: 0.15,
    min_face_quality: 0.5,
    min_face_acceptability: 0.15,
    min_face_frontality: 70,
    max_face_mask_probability: 0.5,
    image_illumination_control: 50,
    max_face_size_pct: 0.72,
    image_boundary_width_pct: 0.8,
    image_boundary_height_pct: 0.8,
    min_face_width: 100,
    max_face_roll_angle: 45,
    fail_fast: true,
)
```

- **min_face_sharpness**: A threshold for a minimum face sharpness required to pass the validness check. The possible range of face sharpness values is from `0` to `1.0` and the default value is `0.15`. The relevant feedback is `FACE_SHARPNESS_POOR`.
- **min_face_quality**: A threshold for a minimum face quality required to pass the validness check. The possible range of face quality values is from `0` to `1.0`

and the default value is `0.5`. The relevant feedback is `FACE_QUALITY_POOR`.

- **min_face_acceptability**: A threshold for a minimum face acceptability required for a face in an image to pass the validness check. The possible range of acceptability values is from `0` to `1.0` and the default value is `0.15`. The relevant feedback is `FACE_ACCEPTABILITY_POOR`.
- **min_face_frontality**: A threshold for a minimum face frontality required for a face in an image to pass the validness check. The possible range of frontality values is from `0` to `100` and the default value is `70`. The relevant feedback is `FACE_FRONTALITY_POOR`.
- **max_face_mask_probability**: A threshold of a face wearing mask. The face mask probability should be below the threshold to pass the validness check. The possible range of mask probability values is from `0` to `1.0` and the default value is `0.5`. The relevant feedback is `FACE_MASK_FOUND`.
- **image_illumination_control**: A threshold for illumination control for an image. This is used internally to determine if the image has the appropriate illumination to pass the validness check. The possible range of illumination control values is from `0` to `100` and the default value is `50`. The relevant feedback are `IMG_LIGHTING_DARK` and `IMG_LIGHTING_BRIGHT`.
- **max_face_size_pct**: A percentage threshold for a maximum face height required in an image to pass the validness check. The possible range of face size values is from `0.5` to `1.0` and the default value is `0.72`. The relevant feedback is `FACE_SIZE_LARGE`.
- **image_boundary_width_pct**: A width percentage threshold to define a boundary where a face should be positioned in an image. The possible range is from `0` to `1.0`. The default value is `0.8`, i.e., the face should be positioned within the central `80%` region of the image width. The relevant feedback are `FACE_POS_LEFT` and `FACE_POS_RIGHT`.
- **image_boundary_height_pct**: A height percentage threshold to define a boundary where a face should be positioned in an image. The possible range is from `0` to `1.0`. The default value is `0.8`, i.e., the face should be positioned within the central `80%` region of the image height. The relevant feedback are `FACE_POS_HIGH` and `FACE_POS_LOW`.
- **min_face_size**: The minimum face width is the minimum absolute width and height of the face, in order for the face to be considered valid. This is measured in pixels and the default value is `100`. For Image source `mobile`, the default value is `175`. The relevant feedback is `FACE_RES_LOW`.

- **max_face_roll_angle**: The maximum absolute angle in degrees that the face may be rotated around its forward axis. The possible range is from `0` to `180` degrees and the default value is `45`. The relevant feedback is `FACE_ROLL_ANGLE_EXCEEDED`.
- **fail_fast**: If set to `false`, `CheckValidness` will perform all validness checks and populate `Validness` object with all feedback checks. If set to true, it will return with a feedback as soon as a validness check fails. The default value is `true`.

> For the liveness feature, the `TOO_MANY_FACES` error is also returned for a given face in an image with multiple faces if a face's expanded bounding box contains another face's bounding box. Visually, if the faces are close in an image, the liveness validness can report `TOO_MANY_FACES` feedback.

Process Full Image Ages_v2 Validness Parameters

An struct providing the validness parameters to pass into the `Process Full Image` function for age estimation. The values in the following code block are the defaults, you do not need to provide them.

C#

```
var ageValidnessParameters = new
pb.ProcessFullImageRequest.Types.AgeValidnessParameters {
    MinFaceSharpness = 0.15,
    MinFaceQuality = 0.5,
    MinFaceAcceptability = 0.15,
    MinFaceFrontality = 60,
    MaxFaceMaskProbibility = 0.5,
    ImageIlluminationControl = 50,
    MaxFaceSizePct = 1.0,
    ImageBoundaryWidthPct = 1.0,
    ImageBoundaryHeightPct = 1.0,
    MinFaceSize = 100,
    FailFast=false
};
```

C++

```
const auto age_validness_params =
std::make_shared<ProcessFullImageAgeValidnessParameters>();
age_validness_params->set_min_face_sharpness(0.15);
age_validness_params->set_min_face_quality(0.5);
age_validness_params->set_min_face_acceptability(0.15);
age_validness_params->set_min_face_frontality(60);
age_validness_params->set_max_face_mask_probability(0.5);
age_validness_params->set_image_illumination_control(50);
age_validness_params->set_min_face_size(100);
age_validness_params->set_max_face_size_pct(1.0);
age_validness_params->set_image_boundary_width_pct(1.0);
age_validness_params->set_image_boundary_height_pct(1.0);
age_validness_params->set_fail_fast(true);
```

Go

```
var minFaceSharpness float32 = 0.15
var minFaceQuality float32 = 0.5
var minFaceAcceptability float32 = 0.15
var minFaceFrontality int32 = 60
var maxFaceMaskProbibility float32 = 0.5
var imageIlluminationControl int32 = 50
var minFaceSize int32 = 100
var maxFaceSizePct float32 = 1.0
var imageBoundaryWidthPct float32 = 1.0
var imageBoundaryHeightPct float32 = 1.0
var failFast bool = false

ageValidnessParameters :=
&pb.ProcessFullImageRequest_AgeValidnessParameters{
        MinFaceSharpness: &minFaceSharpness,
    MinFaceQuality: &minFaceQuality,
    MinFaceAcceptability: &minFaceAcceptability,
    MinFaceFrontality: &minFaceFrontality,
    MaxFaceMaskProbibility: &maxFaceMaskProbibility,
    ImageIlluminationControl: &ImageIlluminationControl,
    MinFaceSize: &minFaceSize,
    MaxFaceSizePct: &maxFaceSizePct,
    ImageBoundaryWidthPct: &imageBoundaryWidthPct,
    ImageBoundaryHeightPct: &imageBoundaryHeightPct,
        FailFast: &failFast,
}
```

Java

```
ProcessFullImageRequest.AgeValidnessParameters ageParams =
ProcessFullImageRequest
    .AgeValidnessParameters.
    newBuilder().
        .setMinFaceSharpness(0.15)
        .setMinFaceQuality(0.5)
        .setMinFaceAcceptability(0.15)
        .setMinFaceFrontality(60)
        .setMaxFaceMaskProbibility(0.5)
        .setImageIlluminationControl(50)
        .setMinFaceSize(100)
        .setMaxFaceSizePct(1.0)
        .setImageBoundaryWidthPct(1.0)
        .setImageBoundaryHeightPct(1.0)
        .setFailFast(true)
        .build();
```

JSON

```
{
    "min_face_sharpness": 0.15,
    "min_face_quality": 0.5,
    "min_face_acceptability": 0.15,
    "min_face_frontality": 60,
    "max_face_mask_probability": 0.5,
    "image_illumination_control": 50,
    "min_face_size": 100,
    "max_face_size_pct": 1.0,
    "image_boundary_width_pct": 1.0,
    "image_boundary_height_pct": 1.0,
    "fail_fast": true
}
```

Node

```
let ageValidnessParameters = new
ProcessFullImageAgeValidnessParameters()
    .setMinFaceSharpness(0.15)
    .setMinFaceQuality(0.5)
    .setMinFaceAcceptability(0.15)
    .setMinFaceFrontality(60)
    .setMaxFaceMaskProbibility(0.5)
    .setImageIlluminationControl(50)
    .setMinFaceSize(100)
    .setMaxFaceSizePct(1.0)
    .setImageBoundaryWidthPct(1.0)
    .setImageBoundaryHeightPct(1.0)
    .setFailFast(true);
```

Python

```
age_validness_params =
processor.ProcessFullImageAgeValidnessParameters(
    min_face_sharpness=0.15,
    min_face_quality=0.5,
    min_face_acceptability=0.15,
    min_face_frontality=60,
    max_face_mask_probability=0.5,
    image_illumination_control=50,
    min_face_size=100,
    max_face_size_pct=1.0,
    image_boundary_width_pct=1.0,
    image_boundary_height_pct=1.0
    fail_fast=True,
)
```

Ruby

```
age_validness_params = ProcessFullImageAgeValidnessParameters.new(
    min_face_sharpness: 0.15,
    min_face_quality: 0.5,
    min_face_acceptability: 0.15,
    min_face_frontality: 60,
    max_face_mask_probability: 0.5,
    image_illumination_control: 50,
    min_face_size: 100,
    max_face_size_pct: 1.0,
    image_boundary_width_pct: 1.0,
    image_boundary_height_pct: 1.0,
    fail_fast: true,
)
```

- **min_face_sharpness**: A threshold for a minimum face sharpness required to pass the validness check. The possible range of face sharpness values is from `0` to `1.0` and the default value is `0.15`. The relevant feedback is `FACE_SHARPNESS_POOR`
- **min_face_quality**: A threshold for a minimum face quality required to pass the validness check. The possible range of face quality values is from `0` to `1.0` and the default value is `0.5`. The relevant feedback is `FACE_QUALITY_POOR`.
- **min_face_acceptability**: A threshold for a minimum face acceptability required for a face in an image to pass the validness check. The possible range of acceptability values is from `0` to `1.0` and the default value is `0.15`. The relevant feedback is `FACE_ACCEPTABILITY_POOR`.
- **min_face_frontality**: A threshold for a minimum face frontality required for a face in an image to pass the validness check. The possible range of frontality values is from `0` to `100` and the default value is `60`. The relevant feedback is `FACE_FRONTALITY_POOR`.
- **max_face_mask_probability**: A threshold of a face wearing mask. The face mask probability should be below the threshold to pass the validness check. The possible range of mask probability values is from `0` to `1.0` and the default value is `0.5`. The relevant feedback is `FACE_MASK_FOUND`.
- **image_illumination_control**: A threshold for illumination control for an image. This is used internally to determine if the image has the appropriate illumination to pass the validness check. The possible range of illumination control values is from `0` to `100` and the default value is `50`. The relevant feedback are `IMG_LIGHTING_DARK` and `IMG_LIGHTING_BRIGHT`.

- **min_face_size**: The minimum face width is the minimum absolute width and height of the face, in order for the face to be considered valid. This is measured in pixels and the default value is `100`. The relevant feedback is `FACE_RES_LOW`.
- **max_face_size_pct**: A percentage threshold for a maximum face height required in an image to pass the validness check. The possible range of face size values is from `0` to `1.0` and the default value is `1.0`. The relevant feedback is `FACE_SIZE_LARGE`.
- **image_boundary_width_pct**: A width percentage threshold to define a boundary where a face should be positioned in an image. The possible range is from `0` to `1.0`. The default value is `1.0`, i.e., the face should be positioned within the central `100%` region of the image width. The relevant feedback are `FACE_POS_LEFT` and `FACE_POS_RIGHT`.
- **image_boundary_height_pct**: A height percentage threshold to define a boundary where a face should be positioned in an image. The possible range is from `0` to `1.0`. The default value is `1.0`, i.e., the face should be positioned within the central `100%` region of the image height. The relevant feedback are `FACE_POS_HIGH` and `FACE_POS_LOW`.
- **fail_fast**: If set to `false`, `CheckValidness` will perform all validness checks and populate `Validness` object with all feedback checks. If set to true, it will return with a feedback as soon as a validness check fails. The default value is `true`.

Process Full Image Deepfake Validness Parameters

An struct providing the validness parameters to pass into the `Process Full Image` ↗ function for deepfake detection. The values in the following code block are the defaults, you do not need to provide them.

C#

```
var validnessParameters = new
pb.ProcessFullImageRequest.Types.DeepfakeValidnessParameters {
    MinFaceQuality = 0.5,
    MinFaceAcceptability = 0.15,
    MinFaceSize = 50,
    FailFast=false
};
```

C++

```
const auto validness_params =
std::make_shared<ProcessFullImageDeepfakeValidnessParameters>();
validness_params->set_min_face_quality(0.5);
validness_params->set_min_face_acceptability(0.15);
validness_params->set_min_face_size(50);
validness_params->set_fail_fast(true);
```

Go

```
var minFaceQuality float32 = 0.5
var minFaceAcceptability float32 = 0.15
var minFaceSize int32 = 50
var failFast bool = false

validnessParameters :=
&pb.ProcessFullImageRequest_DeepfakeValidnessParameters{
    MinFaceQuality: &minFaceQuality,
    MinFaceAcceptability: &minFaceAcceptability,
    MinFaceSize: &minFaceSize,
        FailFast: &failFast,
}
```

Java

```
ProcessFullImageRequest.DeepfakeValidnessParameters validnessParams
= ProcessFullImageRequest
    .DeepfakeValidnessParameters.
    newBuilder().
        .setMinFaceQuality(0.5)
        .setMinFaceAcceptability(0.15)
        .setMinFaceSize(50)
        .setFailFast(true)
        .build();
```

JSON

```
{
    "min_face_quality": 0.5,
    "min_face_acceptability": 0.15,
    "min_face_size": 50,
    "fail_fast": true
}
```

Node

```
let validnessParameters = new
ProcessFullImageDeepfakeValidnessParameters()
    .setMinFaceQuality(0.5)
    .setMinFaceAcceptability(0.15)
    .setMinFaceSize(50)
    .setFailFast(true);
```

Python

```
validness_params =
processor.ProcessFullImageDeepfakeValidnessParameters(
    min_face_quality=0.5,
    min_face_acceptability=0.15,
    min_face_size=50,
    fail_fast=True,
)
```

Ruby

```
validness_params = ProcessFullImageDeepfakeValidnessParameters.new(
    min_face_quality: 0.5,
    min_face_acceptability: 0.15,
    min_face_size: 50,
    fail_fast: true,
)
```

- **min_face_quality**: A threshold for a minimum face quality required to pass the validness check. The possible range of face quality values is from `0` to `1.0` and the default value is `0.5`. The relevant feedback is `FACE_QUALITY_POOR`.
- **min_face_acceptability**: A threshold for a minimum face acceptability required for a face in an image to pass the validness check. The possible range of acceptability values is from `0` to `1.0` and the default value is `0.15`. The relevant feedback is `FACE_ACCEPTABILITY_POOR`.
- **min_face_size**: The minimum face size is the minimum absolute width and height of the face, in order for the face to be considered valid. This is measured in pixels and the default value is `50`. The relevant feedback is `FACE_RES_LOW`.
- **fail_fast**: If set to `false`, `CheckValidness` will perform all validness checks and populate `Validness` object with all feedback checks. If set to true, it will return with a feedback as soon as a validness check fails. The default value is `true`.

Process Full Image Response

An object representing a response of a `Process Full Image` function call.

- **faces**: a list of `Faces` found in the image.

C#

```
var faces = response.Faces;
```

C++

```
std::vector<paravision::processor::Face> faces = response.faces;
```

Go

```
faces := response.GetFaces()
```

Java

```
List<Face> faces = response.getFacesList();
```

JSON

```
response["faces"]
```

Node

```
var faces = response.getFacesList();
```

Python

```
faces = response.faces
```

Ruby

```
faces = response.faces
```

- **most prominent face idx**: index in the list of faces.

C#

```
var idx = response.MostProminentFaceIdx;
```

C++

```
int idx = response.most_prominent_face_idx;
```

Go

```
idx := response.GetMostProminentFaceIdx()
```

Java

```
int idx = response.getMostProminentFaceIdx();
```

JSON

```
response["most_prominent_face_idx"]
```

Node

```
var idx = response.getMostProminentFaceIdx();
```

Python

```
idx = response.most_prominent_face_idx
```

Ruby

```
idx = response.most_prominent_face_idx
```

Process Identity Document Response

An object representing a response of a `Process Identity Document` function call.

- **data: a** string representation of the json response returned by Regula. Refer to their documentation for more details. ↗

Scoring Parameters

A ScoringParameters object stores specific scoring mode parameters used in embeddings comparison computations.

- **sizes**: A list of integers, lengths of the embeddings to be considered for given scoring mode.

C#

```
var sizes = parameters.Sizes;
```

C++

```
std::vector<int> sizes = parameters.sizes;
```

Go

```
sizes := parameters.GetSizes()
```

Java

```
List<int> sizes = parameters.getSizes();
```

JSON

```
parameters["sizes"]
```

Node

```
var sizes = parameters.getSizes();
```

Python

```
sizes = parameters.sizes
```

Ruby

```
sizes = parameters.sizes
```

Liveness Validness Thresholds

A LivenessValidnessThresholds object stores default liveness validness thresholds. Refer Process Full Image Liveness Validness Parameters for more details on

parameters.

C#

```
float sharpness = thresholds.MinFaceSharpness;
float quality = thresholds.MinFaceQuality;
float acceptability = thresholds.MinFaceAcceptability;
int frontality = thresholds.MinFaceFrontality;
float mask = thresholds.MaxFaceMaskProbability;
int illumination = thresholds.ImageIlluminationControl;
float faceSizePct = thresholds.MaxFaceSizePct;
float boundaryWidthPct = thresholds.ImageBoundaryWidthPct;
float boundaryHeightPct = thresholds.ImageBoundaryHeightPct;
int mobileMinFaceSize = thresholds.MobileMinFaceSize;
int webCamMinFaceSize = thresholds.WebcamMinFaceSize;
int rollAngle = thresholds.MaxFaceRollAngle;
bool failFast = thresholds.FailFast;
```

C++

```
float sharpness = thresholds.min_face_sharpness();
float quality = thresholds.min_face_quality();
float acceptability = thresholds.min_face_acceptability();
int frontality = thresholds.min_face_frontality();
float mask = thresholds.max_face_mask_probability();
int illumination = thresholds.image_illumination_control();
float face_size_pct = thresholds.max_face_size_pct();
float boundary_width_pct = thresholds.image_boundary_width_pct();
float boundary_height_pct = thresholds.image_boundary_height_pct();
int mobile_min_size = thresholds.mobile_min_face_size();
int webcam_min_size = thresholds.webcam_min_face_size();
int roll_angle = thresholds.max_face_roll_angle();
bool fail_fast = thresholds.fail_fast();
```

Go

```
sharpness := thresholds.GetMinFaceSharpness()
quality := thresholds.GetMinFaceQuality()
acceptability := thresholds.GetMinFaceAcceptability()
frontality := thresholds.GetMinFaceFrontality()
mask := thresholds.GetMaxFaceMaskProbability()
illumination := thresholds.GetImageIlluminationControl()
faceSizePct := thresholds.GetMaxFaceSizePct()
boundaryWidthPct := thresholds.GetImageBoundaryWidthPct()
boundaryHeightPct := thresholds.GetImageBoundaryHeightPct()
mobileMinSize := thresholds.GetMobileMinFaceSize()
webcamMinSize := thresholds.GetWebcamMinFaceSize()
rollAngle := thresholds.GetMaxFaceRollAngle()
failFast1 := thresholds.GetFailFast()
```

Java

```
float sharpness = thresholds.getMinFaceSharpness();
float quality = thresholds.getMinFaceQuality();
float acceptability = thresholds.getMinFaceAcceptability();
int frontality = thresholds.getMinFaceFrontality();
float mask = thresholds.getMaxFaceMaskProbability();
int illumination = thresholds.getImageIlluminationControl();
float faceSizePct = thresholds.getMaxFaceSizePct();
float boundaryWidthPct = thresholds.getImageBoundaryWidthPct();
float boundaryHeightPct = thresholds.getImageBoundaryHeightPct();
int mobileMinSize = thresholds.getMobileMinFaceSize();
int webcamMinSize = thresholds.getWebcamMinFaceSize();
int rollAngle = thresholds.getMaxFaceRollAngle();
boolean failFast = thresholds.getFailFast();
```

JSON

```
thresholds["min_face_sharpness"]
thresholds["min_face_quality"]
thresholds["min_face_acceptability"]
thresholds["min_face_frontality"]
thresholds["max_face_mask_probability"]
thresholds["image_illumination_control"]
thresholds["max_face_size_pct"]
thresholds["image_boundary_width_pct"]
thresholds["image_boundary_height_pct"]
thresholds["mobile_min_face_size"]
thresholds["webcam_min_face_size"]
thresholds["max_face_roll_angle"]
thresholds["fail_fast"]
```

Node

```
let sharpness = thresholds.getMinFaceSharpness();
let quality = thresholds.getMinFaceQuality();
let acceptability = thresholds.getMinFaceAcceptability();
let frontality = thresholds.getMinFaceFrontality();
let mask = thresholds.getMaxFaceMaskProbability();
let illumination = thresholds.getImageIlluminationControl();
let faceSizePct = thresholds.getMaxFaceSizePct();
let boundaryWidthPct = thresholds.getImageBoundaryWidthPct();
let boundaryHeightPct = thresholds.getImageBoundaryHeightPct();
let mobileMinSize = thresholds.getMobileMinFaceSize();
let webcamMinSize = thresholds.getWebcamMinFaceSize();
let rollAngle = thresholds.getMaxFaceRollAngle();
let failFast = thresholds.getFailFast();
```

Python

```
sharpness = thresholds.min_face_sharpness
quality = thresholds.min_face_quality
acceptability = thresholds.min_face_acceptability
frontality = thresholds.min_face_frontality
mask = thresholds.max_face_mask_probability
illumination = thresholds.image_illumination_control
face_size_pct = thresholds.max_face_size_pct
boundary_width_pct = thresholds.image_boundary_width_pct
boundary_height_pct = thresholds.image_boundary_height_pct
mobile_min_face = thresholds.mobile_min_face_size
webcam_min_face = thresholds.webcam_min_face_size
roll_angle = thresholds.max_face_roll_angle
fail_fast = thresholds.fail_fast
```

Ruby

```
sharpness = thresholds.min_face_sharpness
quality = thresholds.min_face_quality
acceptability = thresholds.min_face_acceptability
frontality = thresholds.min_face_frontality
mask = thresholds.max_face_mask_probability
illumination = thresholds.image_illumination_control
size_pct = thresholds.max_face_size_pct
width_pct = thresholds.image_boundary_width_pct
height_pct = thresholds.image_boundary_height_pct
mobile_min_size = thresholds.mobile_min_face_size
webcam_min_size = thresholds.webcam_min_face_size
roll_angle = thresholds.max_face_roll_angle
fail_fast = thresholds.fail_fast
```

Ages_V2 Validness Thresholds

An AgesV2ValidnessThresholds object stores default ages_v2 validness thresholds. Refer Process Full Image Ages_v2 Validness Parameters for more details on parameters.

C#

```
float sharpness = thresholds.MinFaceSharpness;
float quality = thresholds.MinFaceQuality;
float acceptability = thresholds.MinFaceAcceptability;
int frontality = thresholds.MinFaceFrontality;
float mask = thresholds.MaxFaceMaskProbability;
int illumination = thresholds.ImageIlluminationControl;
float faceSizePct = thresholds.MaxFaceSizePct;
float boundaryWidthPct = thresholds.ImageBoundaryWidthPct;
float boundaryHeightPct = thresholds.ImageBoundaryHeightPct;
int minFaceSize = thresholds.MinFaceSize;
bool failFast = thresholds.FailFast;
```

C++

```
float sharpness = thresholds.min_face_sharpness();
float quality = thresholds.min_face_quality();
float acceptability = thresholds.min_face_acceptability();
int frontality = thresholds.min_face_frontality();
float mask = thresholds.max_face_mask_probability();
int illumination = thresholds.image_illumination_control();
float face_size_pct = thresholds.max_face_size_pct();
float boundary_width_pct = thresholds.image_boundary_width_pct();
float boundary_height_pct = thresholds.image_boundary_height_pct();
int min_size = thresholds.min_face_size();
bool fail_fast = thresholds.fail_fast();
```

Go

```
sharpness := thresholds.GetMinFaceSharpness()
quality := thresholds.GetMinFaceQuality()
acceptability := thresholds.GetMinFaceAcceptability()
frontality := thresholds.GetMinFaceFrontality()
mask := thresholds.GetMaxFaceMaskProbability()
illumination := thresholds.GetImageIlluminationControl()
faceSizePct := thresholds.GetMaxFaceSizePct()
boundaryWidthPct := thresholds.GetImageBoundaryWidthPct()
boundaryHeightPct := thresholds.GetImageBoundaryHeightPct()
minSize := thresholds.GetMinFaceSize()
failFast := thresholds.GetFailFast()
```

Java

```
float sharpness = thresholds.getMinFaceSharpness();
float quality = thresholds.getMinFaceQuality();
float acceptability = thresholds.getMinFaceAcceptability();
int frontality = thresholds.getMinFaceFrontality();
float mask = thresholds.getMaxFaceMaskProbability();
int illumination = thresholds.getImageIlluminationControl();
float faceSizePct = thresholds.getMaxFaceSizePct();
float boundaryWidthPct = thresholds.getImageBoundaryWidthPct();
float boundaryHeightPct = thresholds.getImageBoundaryHeightPct();
int minSize = thresholds.getMinFaceSize();
boolean failFast = thresholds.getFailFast();
```

JSON

```
thresholds["min_face_sharpness"]
thresholds["min_face_quality"]
thresholds["min_face_acceptability"]
thresholds["min_face_frontality"]
thresholds["max_face_mask_probability"]
thresholds["image_illumination_control"]
thresholds["max_face_size_pct"]
thresholds["image_boundary_width_pct"]
thresholds["image_boundary_height_pct"]
thresholds["min_face_size"]
thresholds["fail_fast"]
```

Node

```
let sharpness = thresholds.getMinFaceSharpness();
let quality = thresholds.getMinFaceQuality();
let acceptability = thresholds.getMinFaceAcceptability();
let frontality = thresholds.getMinFaceFrontality();
let mask = thresholds.getMaxFaceMaskProbability();
let illumination = thresholds.getImageIlluminationControl();
let faceSizePct = thresholds.getMaxFaceSizePct();
let boundaryWidthPct = thresholds.getImageBoundaryWidthPct();
let boundaryHeightPct = thresholds.getImageBoundaryHeightPct();
let minSize = thresholds.getMinFaceSize();
let failFast = thresholds.getFailFast();
```

Python

```
sharpness = thresholds.min_face_sharpness
quality = thresholds.min_face_quality
acceptability = thresholds.min_face_acceptability
frontality = thresholds.min_face_frontality
mask = thresholds.max_face_mask_probability
illumination = thresholds.image_illumination_control
face_size_pct = thresholds.max_face_size_pct
boundary_width_pct = thresholds.image_boundary_width_pct
boundary_height_pct = thresholds.image_boundary_height_pct
min_face = thresholds.min_face_size
fail_fast = thresholds.fail_fast
```

Ruby

```
sharpness = thresholds.min_face_sharpness
quality = thresholds.min_face_quality
acceptability = thresholds.min_face_acceptability
frontality = thresholds.min_face_frontality
mask = thresholds.max_face_mask_probability
illumination = thresholds.image_illumination_control
size_pct = thresholds.max_face_size_pct
width_pct = thresholds.image_boundary_width_pct
height_pct = thresholds.image_boundary_height_pct
min_size = thresholds.min_face_size
fail_fast = thresholds.fail_fast
```

Deepfake Validness Thresholds

A DeepfakeValidnessThresholds object stores default deepfake validness thresholds. Refer Process Full Image Deepfake Validness Parameters for more details on parameters.

C#

```
float quality = thresholds.MinFaceQuality;
float acceptability = thresholds.MinFaceAcceptability;
int minFaceSize = thresholds.MinFaceSize;
bool failFast = thresholds.FailFast;
```

C++

```
float quality = thresholds.min_face_quality();
float acceptability = thresholds.min_face_acceptability();
int min_size = thresholds.min_face_size();
bool fail_fast = thresholds.fail_fast();
```

Go

```
quality := thresholds.GetMinFaceQuality()
acceptability := thresholds.GetMinFaceAcceptability()
minSize := thresholds.GetMinFaceSize()
failFast := thresholds.GetFailFast()
```

Java

```
float quality = thresholds.getMinFaceQuality();
float acceptability = thresholds.getMinFaceAcceptability();
int minSize = thresholds.getMinFaceSize();
boolean failFast = thresholds.getFailFast();
```

JSON

```
thresholds["min_face_quality"]
thresholds["min_face_acceptability"]
thresholds["min_face_size"]
thresholds["fail_fast"]
```

Node

```
let quality = thresholds.getMinFaceQuality();
let acceptability = thresholds.getMinFaceAcceptability();
let minSize = thresholds.getMinFaceSize();
let failFast = thresholds.getFailFast();
```

Python

```
quality = thresholds.min_face_quality
acceptability = thresholds.min_face_acceptability
min_face = thresholds.min_face_size
fail_fast = thresholds.fail_fast
```

Ruby

```
quality = thresholds.min_face_quality
acceptability = thresholds.min_face_acceptability
min_size = thresholds.min_face_size
fail_fast = thresholds.fail_fast
```

Status (C++)

A Status object in C++ representing call status (modeled after `grpc::Status` ↗, extracted not to require gRPC headers installed).

- **code** - A `StatusCode`, numerical value representing error (or lack thereof)
- **error_message** - A string, human-readable error message
- **error_details** - A string, additional error details
- **ok()** - A function, returning boolean depending on the status (`true` if successful), example usage:

```
if(!status.ok())
{
    cerr << "Error: " << status.error_message
         << ": " << status.error_details << endl;
    exit(1);
}
```

StatusCode (C++)

An enum providing statuses for `Status` object

- **OK**
- **CANCELLED**
- **UNKNOWN**
- **INVALID_ARGUMENT**
- **DEADLINE_EXCEEDED**
- **NOT_FOUND**
- **ALREADY_EXISTS**
- **PERMISSION_DENIED**
- **UNAUTHENTICATED**
- **RESOURCE_EXHAUSTED**
- **FAILED_PRECONDITION**
- **ABORTED**
- **OUT_OF_RANGE**
- **UNIMPLEMENTED**
- **INTERNAL**
- **UNAVAILABLE**

- **DATA_LOSS**

Protobuf Compiler

As of release 6.6.0 we advise you use the protobuf compiler `protoc` to generate client libraries for integration with your application. The proto source files are stored here ↗.

The gRPC website ↗ contains a list of all the languages we previously supported for the clients, however this is not a list of all supported compile targets for protoc. If your language isn't there or you need to use a different type of binding (async vs sync) there is likely already a compiler that exists and is widely used. The official gRPC examples are stored here ↗.

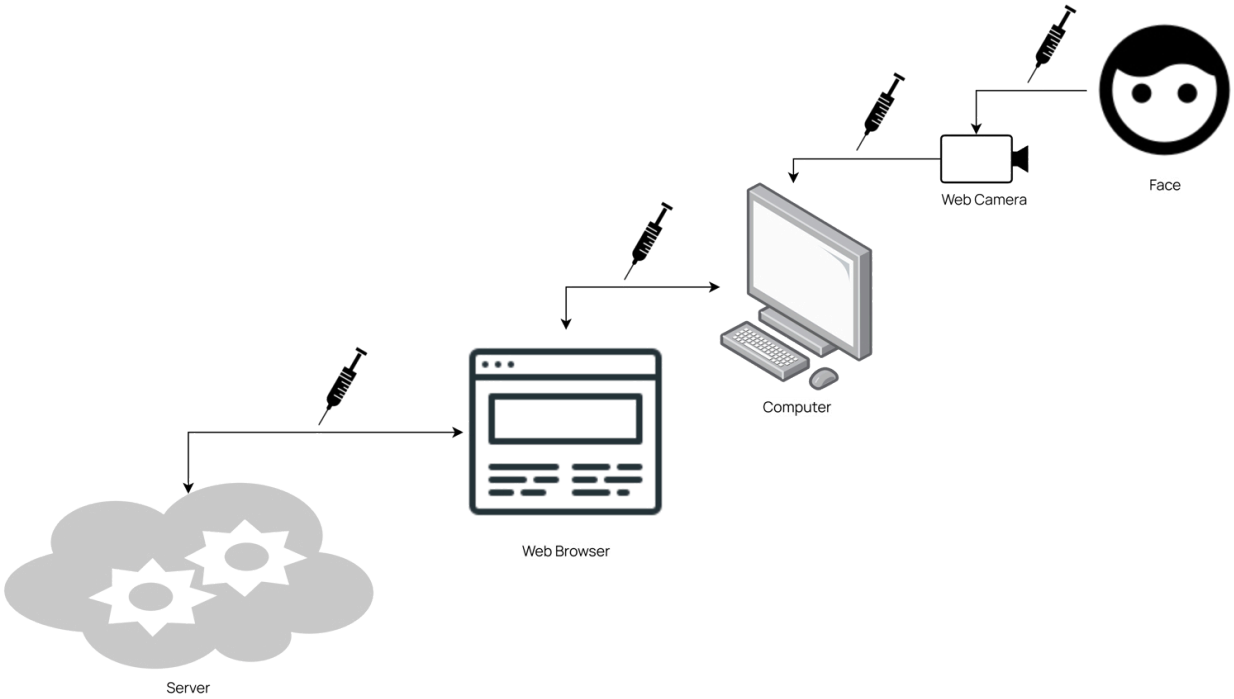
Injection Detection (Experimental)

Injection Detection introduced in Processor v7.9.0 is an experimental feature.

Experimental features may not offer the same level of accuracy or functionality as official features and are subject to change.

Overview

Injection attacks within remote authentication is a rising threat that exposes vulnerabilities in the capturing process of faces used for identification. These types of attacks can include spoofing of the video source, manipulation of the transmission of data, code manipulation and injection, and more.



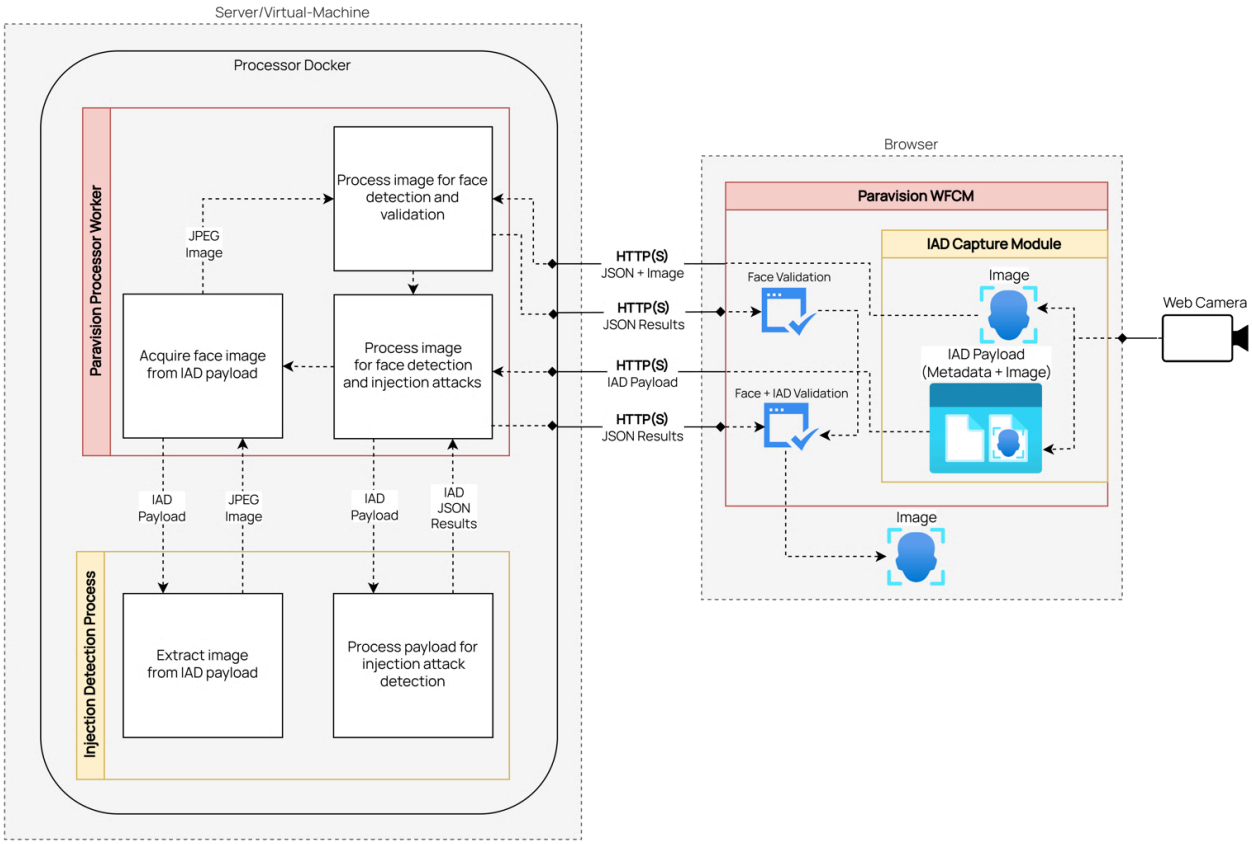
Configuration

To help prevent these types of attacks, the Paravision Processor is now equipped with Injection Attack Detection (IAD) that is available starting from v7.9.0. In order to

minimize resource usage this support is not enabled by default. To enable this support, see PV_INJECTION_DETECTION.

Communication

The Paravision Processor requires a compatible capture client to collect the necessary data needed for injection detected. This data includes a captured face image that adheres to specific validness constraints, and environmental metadata that helps to detect potential injections.



The following capture clients are supported:

- Paravision Web Face Capture Module v1.0.0+

> The following API enhancements are not intended to be utilized directly and are only compatible with the properly captured data acquired from the capture modules above.

Process Full Image

The Process Full Image API has been enhanced to support injection attack detection while being fully backwards compatible with the normal behavior. The additional enhancements are explained below:

```
POST /v7/process_full_image
```

Request

In addition to all the existing support of Request parameters:

outputs[INJECTION_DETECTION]: A new `outputs` option in `INJECTION_DETECTION` that is supplied to process the `injection_detection_payload` for injection attacks. If the `injection_detection_payload` is not given, this option will have no effect.

injection_detection_payload: Base64-encoded bytes of the data bundle captured from the compatible capture clients utilized for injection detection. The payload consists of a face image and environmental metadata. This payload can also be utilized with any of the existing `outputs` options, which will be processed against the face image within the payload.

Response

In addition to all the existing support of Response:

injection_detection_result: A JSON object containing details on the results of the injection detection from the provided `injection_detection_payload`:

- **injection_detected**: A boolean indicating whether an injection attack was detected or not.
- **error**: An optional string value indicating the error, if any, occurred.

Release Notes

API

v7.11.0

June 12, 2025

Engineering Release Notes

- Experimental feature:
  - Introduced support for `gen6.1` face detector, that can be enabled by specifying `gen6.1` for `PV_DETECTION_MODEL` env variable when initializing container. It offers enhanced accuracy, with a trade-off in inferencing speed.
- Internal image optimizations.

v7.10.0

May 20, 2025

Engineering Release Notes

- Improved Liveness2D webcam model.
- Improved deepfake model and expanded the deepfake feature to introduce deepfake types.
  - `deepfake_realness` score at the root of the `Face` is now deprecated. Please use the `deepfake` object of the `Face` which includes the `realness` score and a `deepfake type` (e.g. face swap, synthetic), along with detailed scores of deepfake types.
- Fixed CVE
  - CVE-2025-22869 ↗

v7.9.0

April 11, 2025

Engineering Release Notes

- Updated Liveness SDK version to correctly report v2.0.0
- Bug fix for correctly applying `max_face_size_pct` threshold for liveness and age validness checks.
- Updated `get_metadata` API to report correct `active_embedding_size` when standard scoring mode is used.
- Removed deepfake-only processor images.
- Experimental features:
  - Added integration for Regula document authentication via the `process_identity_document` endpoint. For installation instructions, please see here.
    - gRPC clients are updated for Document Auth feature.
  - Support for injection attack detection with compatible Paravision capture clients:
    - Only supported with Processor-All-IAD Docker image variant.
      - Support turned off by default, can be enabled via PV_INJECTION_DETECTION.
      - API enhancements to `process_full_image` to support processing injection detection, see Process Full Image.
      - A new `processor-all-iad` processor image is introduced for IAD feature.
  - Added ability to count usage of features and generate reports using the endpoint `get_usage_report`.
- Fixed CVEs
  - CVE-2024-26928 ↗
  - CVE-2024-35864 ↗
  - CVE-2024-53140 ↗
  - CVE-2024-53168 ↗
  - CVE-2024-56595 ↗
  - CVE-2024-56598 ↗
  - CVE-2024-56658 ↗
  - CVE-2024-45337 ↗
  - CVE-2024-24790 ↗

- CVE-2023-21400 ↗ (openvino image only)
- CVE-2024-50264 ↗ (openvino image only)
- CVE-2024-53103 ↗ (openvino image only)
- CVE-2024-45338 ↗
- CVE-2023-45288 ↗
- CVE-2024-34156 ↗

v7.8.0

February 3, 2025

Engineering Release Notes

- Update `get_metadata` API to report default validness thresholds for liveness, ages_v2 and deepfake features.

v7.7.1

December 11, 2024

Engineering Release Notes

- Fix for `compare_most_prominent_faces` API
- Internal bug fixes

v7.7.0

December 9, 2024

Engineering Release Notes

- Integrated improved Liveness, age and deepfake models.
- For Liveness feature, it is crucial to specify the image source correct to achieve higher accuracy. Refer usage.
- For the `liveness_validness_parameters` in the `process_full_image` endpoint, `min_face_height_pct`, and `min_face_width` are now deprecated, and `min_face_size`, and `max_face_roll_angle` have been added.

- For the `age_validness_parameters` in the `process_full_image` endpoint, `min_face_width` is now deprecated, and `min_face_size`, `max_face_size_pct`, `image_boundary_width_pct`, `image_boundary_height_pct` have been added.
- For the `deepfake_validness_parameters` in the `process_full_image` endpoint, `min_face_width` is now deprecated, and `min_face_size` has been added.
- In the `validness` response, `height_pct` and `width_pct` are now deprecated, and `face_height_pct`, `face_width_pct`, and `face_roll_angle` have been added. The `face_roll_angle` is applicable to liveness validness only.
- Added `deepfake_images` output option in the `process_full_image` endpoint to output prepared deepfake images.
- Added `liveness_image` output option in the `process_full_image` endpoint to output prepared liveness image.
- Added `ages_v2_image` output option in the `process_full_image` endpoint to output prepared ages_v2 image.
- Added `deepfake_images` input option and `DEEPFAKE` output option in the `process_prepared_face_image` endpoint to run deepfake inference on prepared deepfake images.
- Added `liveness_image` input option and `LIVENESS` output option in the `process_prepared_face_image` endpoint to run liveness inference on a prepared liveness image.
- Added `ages_v2_image` input option in the `process_prepared_face_image` endpoint to run liveness inference on a prepared ages_v2 image.
- gRPC client updates
- Added `PV_LOG_FORMAT` environment variable for custom log formatting. Refer usage.
- Fixed CVEs
  - CVE-2024-45490 ↗
  - CVE-2024-45491 ↗
  - CVE-2024-45492 ↗
  - CVE-2024-6232 ↗

v7.6.1

October 28, 2024

Engineering Release Notes

- Fixed CVE
  - CVE-2022-48522 ↗

v7.5.0

March 15, 2024

Engineering Release Notes

- Introduces the improved age estimation feature for face images
- Added two options `AGES_V2` and `AGES_V2_VALIDNESS` in `process_full_image`
- Added an option `AGES_V2` in `process_prepared_face_image`
- Updated metadata response to add `engine_name`, and removed `recognition_engine_name`, `attributes_engine_name`, `deepfake_engine_name`, and `liveness2d_engine_name`

v7.4.0

February 8, 2024

Engineering Release Notes

- Introduces the deepfake feature to run deepfake analysis on face images.
- A `DEEPFAKE` option is added to `process_full_image` to run deepfake analysis. The output `deepfake_realness` score reports the probability of an unaltered face image.

v7.3.0

January 29, 2024

Engineering Release Notes

- Renamed processAlignedFaceImage API to processPreparedFaceImage API.
- Updated processFullImage API to include an option ATTRIBUTES_IMAGES to return the required images for processPreparedFaceImage.
- Bug fixes for attributes options in processPreparedFaceImage.
- Bug fix for compare_most_prominent_face when image with no face is passed.
- Fixed CVEs
  - GHSA-2c7c-3mj9-8fqh
  - CVE-2023-39325
  - CVE-2023-3978
  - CVE-2023-44487
  - GHSA-m425-mq94-257g
  - CVE-2023-44487

v7.2.0

December 5, 2023

Engineering Release Notes

- Updated behavior for environment variable `PV_DETECTION_MODEL` to allow users to specify `mobile` detection model, especially when the liveness feature is enabled. Prevents situations when the Mobile detector was selected implicitly causing faces not to be detected for other purposes.
- A new error code (OTHER) was added to the response message. Addresses a situation when the server encounters an internal failure during liveness estimation and still returns a valid result.
- Added the ability to run as a non-root user. To do so, use the `-u paravision` argument in the `docker run` command.
- Bug fixes

v7.1.0

October 31, 2023

Engineering Release Notes

- Processor with liveness includes optimized model for level 2 mobile capture

v7.0.1

September 27, 2023

- Includes bug fix for the HTTP endpoints.
- Add support for PV_CORS_ORIGIN.

v7.0.0

August 31, 2023

- Support for Generation 6 (Gen6) of Paravision's Recognition, Attributes and Liveness SDKs and models.
- Metadata response now includes additional version details for Recognition, Attributes and Liveness SDK and models. See Metadata response in the Types page for more information.
- Weight and Bias have been removed from the Metadata response
- Confidence in scores request and response has been removed. Please use Match score.
- The default for environment variable PV_SCORING_MODE is set to `enhanced` by default. It was set to `standard` in previous generation 5.

C#

v7.10.0

May 20, 2025

Engineering Release Notes

- Updates for new Deepfake Face object.

v7.9.0

April 11, 2025

Engineering Release Notes

- Experimental feature:
  - Added `ProcessIdentityDocument` function for identity document authentication with Regula.
  - Added `GetUsageReport` function for generate usage report of features transacted using Processor API.

v7.8.0

February 3, 2025

Engineering Release Notes

- Update `GetMetadata` to report default validness thresholds for liveness, ages_v2 and deepfake features.

v7.7.0

December 9, 2024

Engineering Release Notes

- Updated `ProcessPreparedFaceImage` to add an argument `deepfake_images` returned from `ProcessFullImage` when `DEEPFAKE_IMAGES` option is passed. Refer usage.
- Updated `ProcessPreparedFaceImage` to add an argument `liveness_image` returned from `ProcessFullImage` when `LIVENESS_IMAGE` option is passed. Refer usage.
- Updated `ProcessPreparedFaceImage` to add an argument `ages_v2_image` returned from `ProcessFullImage` when `AGES_V2_IMAGE` option is passed. Refer usage.

v7.6.0

May 8, 2024

Engineering Release Notes

- Updated `ProcessFullImage` requires deepfake_validness and image_source request parameters. Refer usage ↗.

v7.5.0

March 15, 2024

Engineering Release Notes

- Updates for the age estimation feature.
- Updated `ProcessFullImage` requires age validness as a request parameter. Refer usage ↗.

v7.4.0

February 8, 2024

Engineering Release Notes

- Updates for the deepfake feature

v7.3.0

January 29, 2024

- Renamed ProcessAlignedFaceImage to ProcessPreparedFaceImage.
- Updated ProcessPreparedFaceImage to add an argument to accept attributes_images returned from ProcessFullImage when ATTRIBUTES_IMAGES option is passed.

v7.0.0

August 31, 2023

- Compatibility with Gen6 Processor API
- Support API protocol v7

C++

v7.10.0

May 20, 2025

Engineering Release Notes

- Updates for new Deepfake Face object.

v7.9.0

April 11, 2025

Engineering Release Notes

- Experimental feature:
  - Added `ProcessIdentityDocument` function for identity document authentication with Regula.
  - Added `GetUsageReport` function for generate usage report of features transacted using Processor API.

v7.8.0

February 3, 2025

Engineering Release Notes

- Update `GetMetadata` to report default validness thresholds for liveness, ages_v2 and deepfake features.

v7.7.0

December 9, 2024

Engineering Release Notes

- Updated `ProcessPreparedFaceImage` to add an argument `deepfake_images` returned from `ProcessFullImage` when `DEEPFAKE_IMAGES` option is passed. Refer usage.
- Updated `ProcessPreparedFaceImage` to add an argument `liveness_image` returned from `ProcessFullImage` when `LIVENESS_IMAGE` option is passed.

Refer usage.

- Updated `ProcessPreparedFaceImage` to add an argument `ages_v2_image` returned from `ProcessFullImage` when `AGES_V2_IMAGE` option is passed. Refer usage

v7.6.0

May 8, 2024

Engineering Release Notes

- Updated `ProcessFullImage` requires deepfake validness and image source as request parameters. Refer usage ↗.

v7.5.0

March 15, 2024

Engineering Release Notes

- Updates for the age estimation feature.
- Updated `ProcessFullImage` requires age validness as a request parameter. Refer usage ↗.

v7.4.0

February 8, 2024

Engineering Release Notes

- Updates for the deepfake feature

v7.3.0

January 29, 2024

- Renamed ProcessAlignedFaceImage to ProcessPreparedFaceImage.
- Updated ProcessPreparedFaceImage to add an argument to accept attributes_images returned from ProcessFullImage when ATTRIBUTES_IMAGES option is passed.

v7.0.0

August 31, 2023

- Compatibility with Gen6 Processor API
- Support API protocol v7

Go

v7.10.0

May 20, 2025

Engineering Release Notes

- Updates for new Deepfake Face object.

v7.9.0

April 11, 2025

Engineering Release Notes

- Experimental feature:
  - Added `ProcessIdentityDocument` function for identity document authentication with Regula.
  - Added `GetUsageReport` function for generate usage report of features transacted using Processor API.

v7.8.0

February 3, 2025

Engineering Release Notes

- Update `GetMetadata` to report default validness thresholds for liveness, ages_v2 and deepfake features.

v7.7.0

December 9, 2024

Engineering Release Notes

- Updated `ProcessPreparedFaceImage` to add an argument `deepfake_images` returned from `ProcessFullImage` when `DEEPFAKE_IMAGES` option is passed. Refer usage.
- Updated `ProcessPreparedFaceImage` to add an argument `liveness_image` returned from `ProcessFullImage` when `LIVENESS_IMAGE` option is passed. Refer usage.
- Updated `ProcessPreparedFaceImage` to add an argument `ages_v2_image` returned from `ProcessFullImage` when `AGES_V2_IMAGE` option is passed. Refer usage.

v7.6.0

May 8, 2024

Engineering Release Notes

- Updated `ProcessFullImage` requires deepfake validness and image source as request parameters. Refer usage ↗.

v7.5.0

March 15, 2024

Engineering Release Notes

- Updates for the age estimation feature.
- Updated `ProcessFullImage` requires age validness as a request parameter. Refer usage ↗.

v7.4.0

February 8, 2024

Engineering Release Notes

- Updates for the deepfake feature

v7.3.0

January 29, 2024

- Renamed ProcessAlignedFaceImage to ProcessPreparedFaceImage.
- Updated ProcessPreparedFaceImage to add an argument to accept attributes_images returned from ProcessFullImage when ATTRIBUTES_IMAGES option is passed.

v7.0.0

August 31, 2023

- Compatibility with Gen6 Processor API
- Support API protocol v7

HTTP

v7.10.0

May 20, 2025

Engineering Release Notes

- Updates for new Deepfake Face object.

v7.9.0

April 11, 2025

Engineering Release Notes

- Experimental feature:
  - Added `/v7/process_identity_document` function for identity document authentication with Regula.
  - Added `/v7/get_usage_report` function for generate usage report of features transacted using Processor API.

v7.8.0

February 3, 2025

Engineering Release Notes

- Update `/v7/get_metadata` to report default validness thresholds for liveness, ages_v2 and deepfake features.

v7.7.0

December 9, 2024

Engineering Release Notes

- Updated `/v7/process_prepared_face_image` to add an argument `deepfake_images` returned from `/v7/process_full_image` when `DEEPFAKE_IMAGES` option is passed. Refer usage.
- Updated `/v7/process_prepared_face_image` to add an argument `liveness_image` returned from `/v7/process_full_image` when `LIVENESS_IMAGE` option is passed. Refer usage.
- Updated `/v7/process_prepared_face_image` to add an argument `ages_v2_image` returned from `/v7/process_full_image` when `AGES_V2_IMAGE` option is passed. Refer usage

v7.6.0

May 8, 2024

Engineering Release Notes

- Updated `/v7/process_full_image` to accept deepfake validness and image source parameters. Refer usage ↗.

v7.5.0

March 15, 2024

Engineering Release Notes

- Updates for the age estimation feature.
- Updated `/v7/process_full_image` to accept age validness parameters. Refer usage ↗.

v7.4.0

February 8, 2024

Engineering Release Notes

- Updates for the deepfake feature

v7.3.0

January 29, 2024

- Renamed process_aligned_face_image to process_prepared_face_image.
- Updated process_prepared_face_image to add an argument to accept attributes_images returned from process_full_image when ATTRIBUTES_IMAGES option is passed.

v7.0.0

August 31, 2023

- Compatibility with Gen6 Processor API
- Support API protocol v7

Java

v7.10.0

May 20, 2025

Engineering Release Notes

- Updates for new Deepfake Face object.

v7.9.0

April 11, 2025

Engineering Release Notes

- Experimental feature:
  - Added `processIdentityDocument` function for identity document authentication with Regula.
  - Added `getUsageReport` function for generate usage report of features transacted using Processor API.

v7.8.0

February 3, 2025

Engineering Release Notes

- Update `getMetadata` to report default validness thresholds for liveness, ages_v2 and deepfake features.

v7.7.0

December 9, 2024

Engineering Release Notes

- Updated `processPreparedFaceImage` to add an argument `deepfake_images` returned from `processFullImage` when `DEEPFAKE_IMAGES` option is passed. Refer usage.
- Updated `processPreparedFaceImage` to add an argument `liveness_image` returned from `processFullImage` when `LIVENESS_IMAGE` option is passed. Refer usage.
- Updated `processPreparedFaceImage` to add an argument `ages_v2_image` returned from `processFullImage` when `AGES_V2_IMAGE` option is passed. Refer usage.

v7.6.0

May 8, 2024

Engineering Release Notes

- Updated `processFullImage` requires deepfake validness and image source as request parameters. Refer usage ↗.

v7.5.0

March 15, 2024

Engineering Release Notes

- Updates for the age estimation feature.
- Updated `processFullImage` requires age validness as a request parameter. Refer usage↗.
- Updated `close()` method to accept timeout in seconds.
- Bug fix.

v7.4.0

February 8, 2024

Engineering Release Notes

- Updates for the deepfake feature

v7.3.0

January 29, 2024

- Renamed processAlignedFaceImage to processPreparedFaceImage.
- Updated processPreparedFaceImage to add an argument to accept attributes_images returned from processFullImage when ATTRIBUTES_IMAGES option is passed.

v7.0.0

August 31, 2023

- Compatibility with Gen6 Processor API
- Support API protocol v7

Node

v7.10.0

May 20, 2025

Engineering Release Notes

- Updates for new Deepfake Face object.

v7.9.0

April 11, 2025

Engineering Release Notes

- Experimental feature:
  - Added `processIdentityDocument` function for identity document authentication with Regula.
  - Added `getUsageReport` function for generate usage report of features transacted using Processor API.

v7.8.0

February 3, 2025

Engineering Release Notes

- Update `getMetadata` to report default validness thresholds for liveness, ages_v2 and deepfake features.

v7.7.0

December 9, 2024

Engineering Release Notes

- Updated `processPreparedFaceImage` to add an argument `deepfake_images` returned from `processFullImage` when `DEEPFAKE_IMAGES` option is passed. Refer usage.
- Updated `processPreparedFaceImage` to add an argument `liveness_image` returned from `processFullImage` when `LIVENESS_IMAGE` option is passed. Refer usage.
- Updated `processPreparedFaceImage` to add an argument `ages_v2_image` returned from `processFullImage` when `AGES_V2_IMAGE` option is passed.

Refer usage.

v7.6.0

May 8, 2024

Engineering Release Notes

- Updated `processFullImage` requires deepfake validness and image source as request parameters. Refer usage ↗.

v7.5.0

March 15, 2024

Engineering Release Notes

- Updates for the age estimation feature.
- Updated `processFullImage` requires age validness as a request parameter. Refer usage ↗.

v7.4.0

February 8, 2024

Engineering Release Notes

- Updates for the deepfake feature

v7.3.0

January 29, 2024

- Renamed processAlignedFaceImage to processPreparedFaceImage.
- Updated processPreparedFaceImage to add an argument to accept attributes_images returned from processFullImage when ATTRIBUTES_IMAGES option is passed.

v7.0.0

August 31, 2023

- Compatibility with Gen6 Processor API
- Support API protocol v7

Python

v7.10.0

May 20, 2025

Engineering Release Notes

- Updates for new Deepfake Face object.

v7.9.0

April 11, 2025

Engineering Release Notes

- Experimental feature:
  - Added `process_identity_document` function for identity document authentication with Regula.
  - Added `get_usage_report` function for generate usage report of features transacted using Processor API.

v7.8.0

February 3, 2025

Engineering Release Notes

- Update `get_metadata` to report default validness thresholds for liveness, ages_v2 and deepfake features.

v7.7.0

December 9, 2024

Engineering Release Notes

- Updated `process_prepared_face_image` to add an argument `deepfake_images` returned from `process_full_image` when `DEEPFAKE_IMAGES` option is passed. Refer usage.
- Updated `process_prepared_face_image` to add an argument `liveness_image` returned from `process_full_image` when `LIVENESS_IMAGE` option is passed. Refer usage.
- Updated `process_prepared_face_image` to add an argument `ages_v2_image` returned from `process_full_image` when `AGES_V2_IMAGE` option is passed. Refer usage.

v7.6.0

May 8, 2024

Engineering Release Notes

- Updated `process_full_image` requires deepfake validness and image source as request parameters. Refer usage ↗.

v7.5.0

March 15, 2024

Engineering Release Notes

- Updates for the age estimation feature.
- Updated `process_full_image` requires age validness as a request parameter. Refer usage ↗.

v7.4.0

February 8, 2024

Engineering Release Notes

- Updates for the deepfake feature

v7.3.0

January 29, 2024

- Renamed process_aligned_face_image to process_prepared_face_image.
- Updated process_prepared_face_image to add an argument to accept attributes_images returned from process_full_image when ATTRIBUTES_IMAGES option is passed.

v7.0.0

August 31, 2023

- Compatibility with Gen6 Processor API
- Support API protocol v7

Ruby

v7.10.0

May 20, 2025

Engineering Release Notes

- Updates for new Deepfake Face object.

v7.9.0

April 11, 2025

Engineering Release Notes

- Experimental feature:
  - Added `process_identity_document` function for identity document authentication with Regula.
  - Added `get_usage_report` function for generate usage report of features transacted using Processor API.

v7.8.0

February 3, 2025

Engineering Release Notes

- Update `get_metadata` to report default validness thresholds for liveness, ages_v2 and deepfake features.

v7.7.0

December 9, 2024

Engineering Release Notes

- Updated `process_prepared_face_image` to add an argument `deepfake_images` returned from `process_full_image` when `DEEPFAKE_IMAGES` option is passed. Refer usage.
- Updated `process_prepared_face_image` to add an argument `liveness_image` returned from `process_full_image` when `LIVENESS_IMAGE` option is passed. Refer usage.
- Updated `process_prepared_face_image` to add an argument `ages_v2_image` returned from `process_full_image` when `AGES_V2_IMAGE` option is passed. Refer usage.

v7.6.0

May 8, 2024

Engineering Release Notes

- Updated `process_full_image` requires deepfake validness and image source as request parameters. Refer usage ↗.

v7.5.0

March 15, 2024

Engineering Release Notes

- Updates for the age estimation feature.
- Updated `process_full_image` requires age validness as a request parameter. Refer usage ↗.

v7.4.0

February 8, 2024

Engineering Release Notes

- Updates for the deepfake feature

v7.3.0

January 29, 2024

- Renamed process_aligned_face_image to process_prepared_face_image.
- Updated process_prepared_face_image to add an argument to accept attributes_images returned from process_full_image when ATTRIBUTES_IMAGES option is passed.

v7.0.0

August 31, 2023

- Compatibility with Gen6 Processor API
- Support API protocol v7